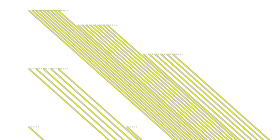


Chapitre 3 : Les statistiques

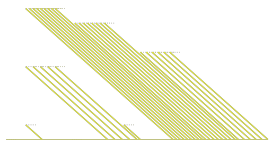
Marie VAUGOYEAU (contact@marievaugoyeau.fr)

Table des matières

1	Avant de commencer	3
2	Prise en main {ggplot2}	3
3	Lois de probabilités	9
3.1	Initiation loi de probabilités : lancer d'une pièce	9
3.1.a	création d'un vecteur contenant le nombre de lancers	9
3.1.b	10 simulations pour chaque valeurs de nb_lancers de pièces	10
3.2	Loto biaisé	11
3.3	Loi uniforme	12
3.3.a	pour un dé équilibré	12
3.3.b	loi de densité uniforme continue	13
3.4	Loi Binomiale	14
3.5	Loi de Poisson	16
3.6	Loi normale	18
3.7	Loi exponentielle	20
4	Statistiques descriptives univariées	21
4.1	Type de données	21
4.2	variable qualitative	22
4.2.a	création d'un graphique en barres	23
4.2.b	création d'un graphique circulaire	24
4.2.c	treemap	25
4.2.d	nuage de mots	26
4.3	variable quantitative	27
4.3.a	valeurs de dispersion	27
4.3.b	nuage de points	28
4.3.c	données de positionnement	29
4.3.d	histogramme avec la moyenne et la médiane	30
4.3.e	Nuage de points avec la moyenne et la médiane	31
4.3.f	nuage de point dispersé	33
4.3.g	quantiles	36
4.3.h	boîte à moustaches	37
4.3.i	visualisation des deciles	39
4.3.j	filtrer sur les valeurs extremes	40



4.3.k	boite à moustaches avec points dispersés	41
5	Statistiques descriptives bivariées	41
5.1	2 variables quantitatives	41
5.1.a	test de Pearson	41
5.1.b	graphique	42
5.2	Une quanti et une quali	42
5.2.a	test du Khi-2	42
5.2.b	représentation graphique avec forme et couleur	43
5.2.c	une ligne par type de véhicule	43
5.2.d	avec des boîtes à moustaches	44
5.3	2 variables qualitatives	45
5.3.a	tableau de contingence	45
5.3.b	carte des points chauds	46
6	Valeurs manquantes	46



1 Avant de commencer

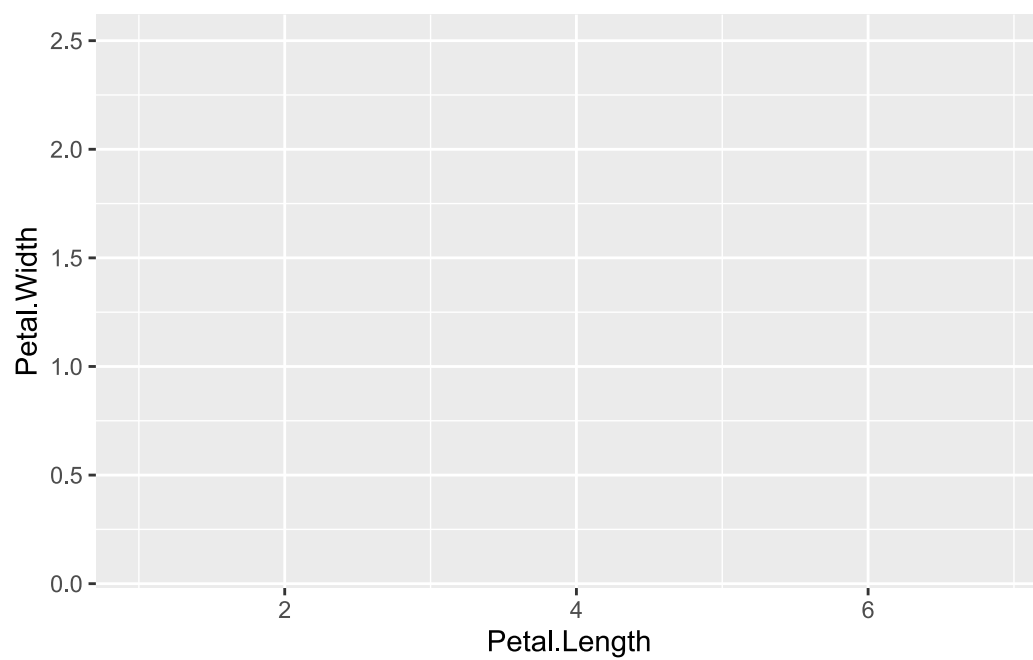
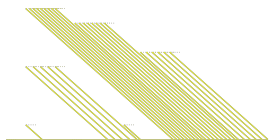
Ce chapitre reprends les codes du chapitre 3 sur **les statistiques** du livre **Langage R et Statistiques** : Initiation à l'analyse de données publié par les **éditions ENI** (2^{ème} édition).

```
library(tidyverse)
```

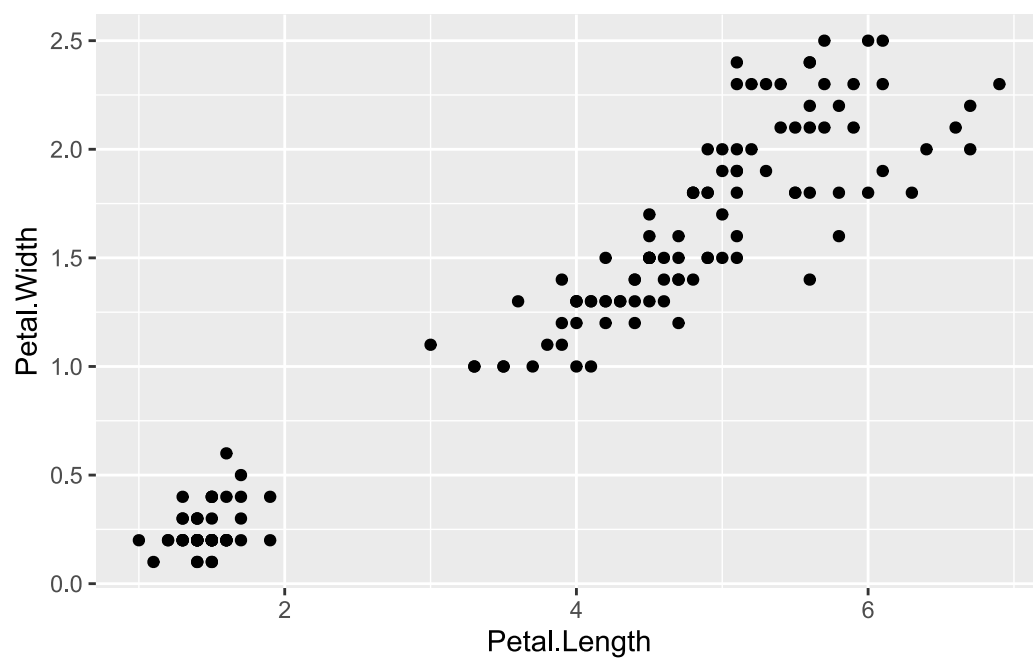
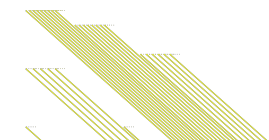
```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1      ✓ readr      2.2.0
✓ forcats    1.0.1      ✓ stringr    1.6.0
✓ ggplot2     4.0.3      ✓ tibble     3.3.1
✓ lubridate  1.9.5      ✓ tidyr      1.3.2
✓ purrr       1.2.2
— Conflicts — tidyverse_conflicts() —
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

2 Prise en main {ggplot2}

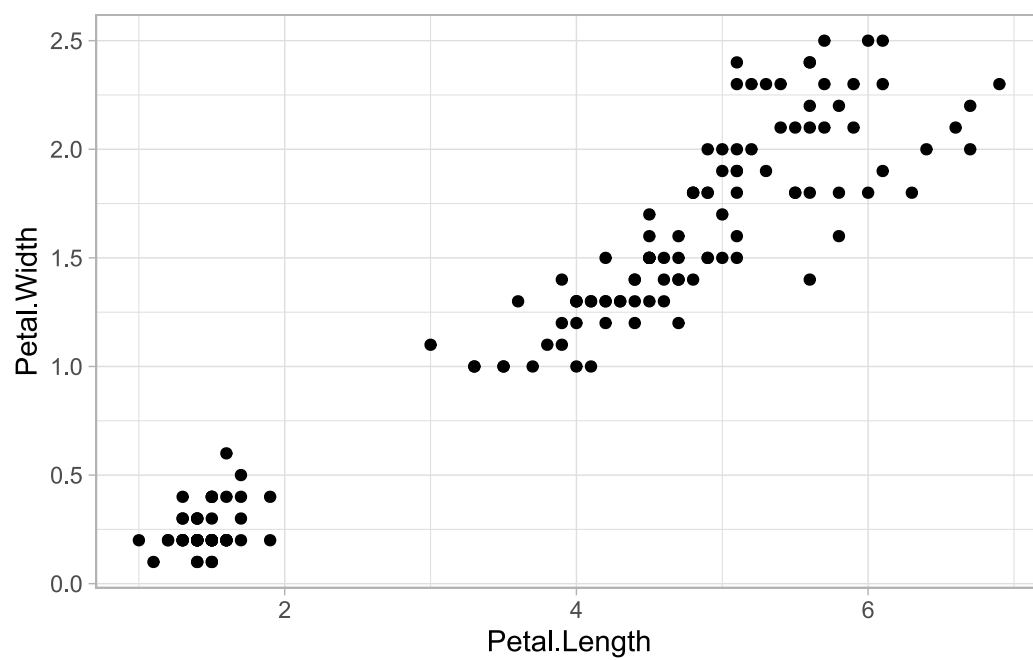
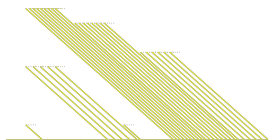
```
ggplot(
  data = iris,
  mapping =
    aes(
      x = Petal.Length,
      y = Petal.Width
    )
)
```



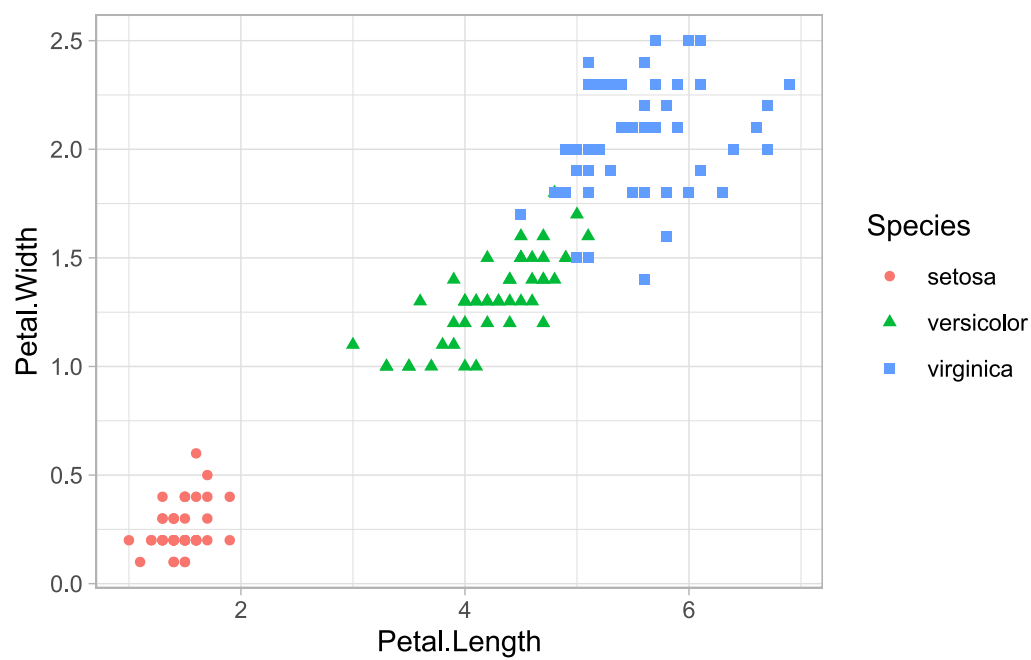
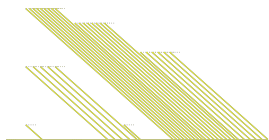
```
ggplot(  
  data = iris  
) +  
  aes(  
    x = Petal.Length,  
    y = Petal.Width  
  ) +  
  geom_point()
```



```
ggplot(  
  data = iris  
) +  
  aes(  
    x = Petal.Length,  
    y = Petal.Width  
  ) +  
  geom_point() +  
  theme_light()
```

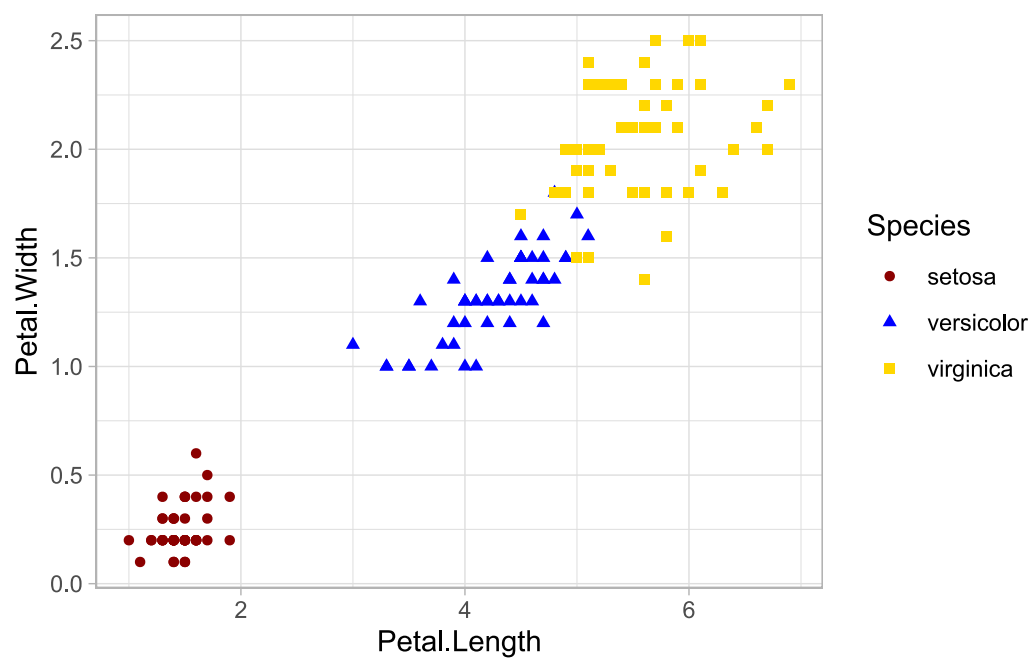
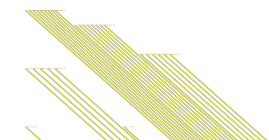


```
ggplot(  
  data = iris  
) +  
  aes(  
    x = Petal.Length,  
    y = Petal.Width,  
    color = Species,  
    shape = Species  
  ) +  
  geom_point() +  
  theme_light()
```

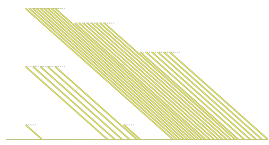


```
ggsave("img/03Rl04N.png", width = 14, height = 10, units = "cm")
```

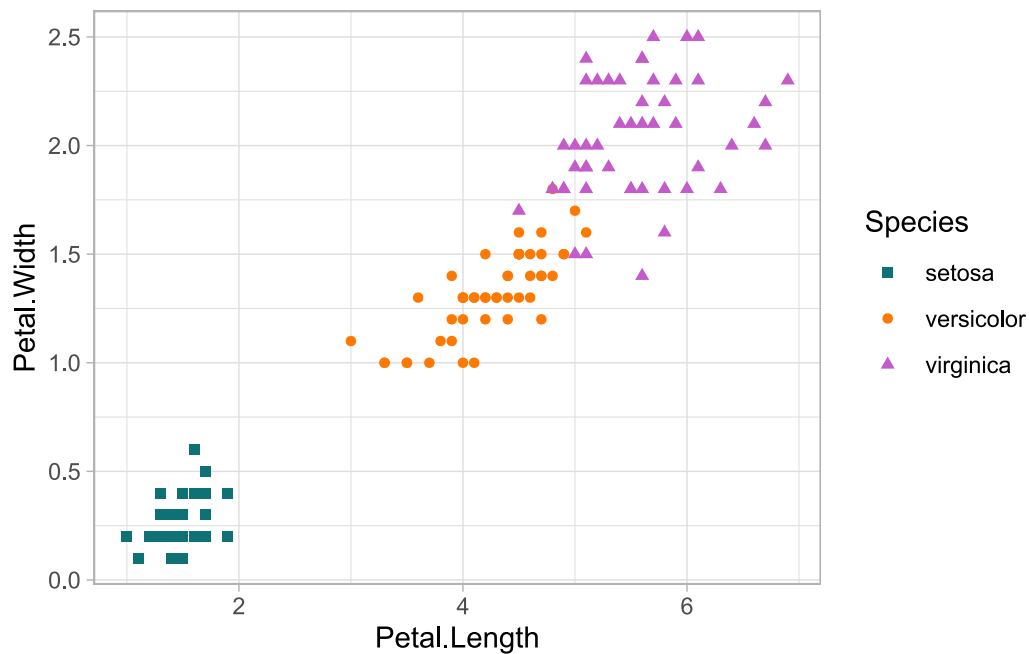
```
ggplot(  
  data = iris  
) +  
  aes(  
    x = Petal.Length,  
    y = Petal.Width,  
    color = Species,  
    shape = Species  
  ) +  
  geom_point() +  
  theme_light() +  
  scale_color_manual(  
    values = c("darkred", "blue", "gold")  
  )  
)
```



```
vecteur_couleur <- c(  
  "setosa" = "#0E7175FF",  
  "versicolor" = "#FD7901FF",  
  "virginica" = "#C35BCAFF"  
)  
  
vecteur_forme <- c(  
  "setosa" = 15,  
  "versicolor" = 16,  
  "virginica" = 17  
)  
  
ggplot(  
  data = iris  
) +  
  aes(  
    x = Petal.Length,  
    y = Petal.Width,  
    color = Species,  
    shape = Species  
  ) +  
  geom_point() +  
  theme_light() +  
  scale_color_manual(  
    values = vecteur_couleur,  
    shape = vecteur_forme  
  )
```

```
values = vecteur_couleur
) +
scale_shape_manual(
  values = vecteur_forme
)
```



```
ggsave("img/03Rl05Nbis.png", width = 14, height = 10, units = "cm")
```

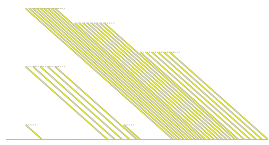
3 Lois de probabilités

3.1 Initiation loi de probabilités : lancer d'une pièce

Pour illustrer le comportement des lois de probabilités autour des grands nombres, je vais représenter le nombre de faces pour un nombre croissant de lancers de pièces.

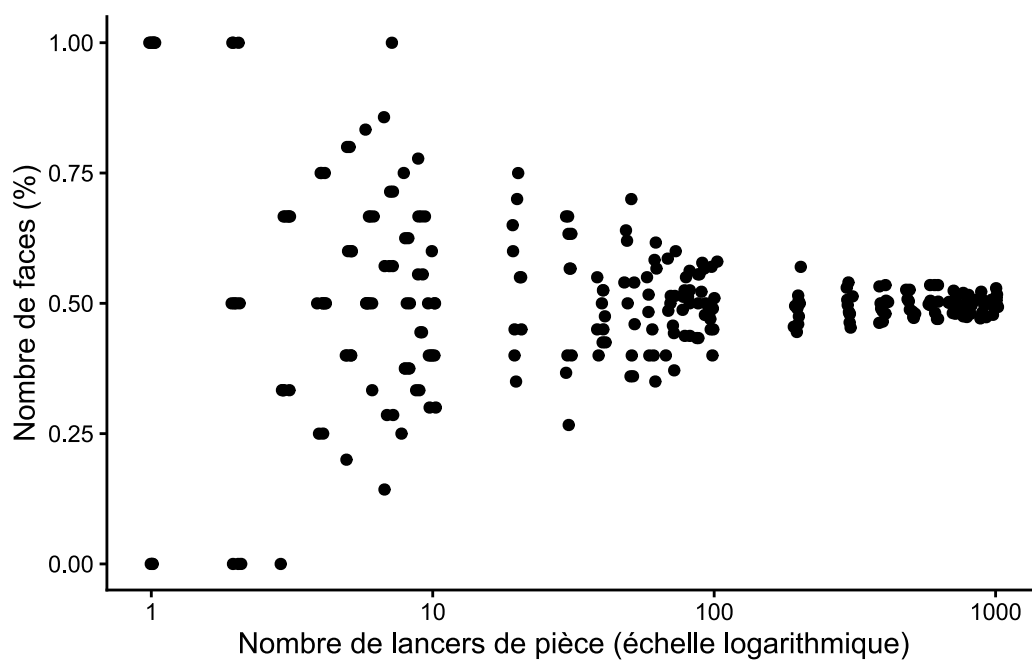
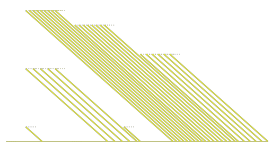
3.1.a création d'un vecteur contenant le nombre de lancers

```
nb_lancers <- c(
  1:9, # 1, 2, 3... 9
  seq(from = 10, to = 100, by = 10), # 10, 20... 100
  seq(from = 200, to = 1000, by = 100) # 200, 300... 1000
)
```



3.1.b 10 simulations pour chaque valeurs de nb_lancers de pièces

```
purrr::map(
  c(1:10),
  ~ bind_rows(
    bind_cols(
      nb_lancers = nb_lancers,
      nb_lancers |>
        purrr::map(
          ~ rbinom(.x, 1, 0.5)
        ) |>
        purrr::map(sum) |>
        purrr::map(as_tibble) |>
        bind_rows() |>
        rename(nb_faces = value)
    )
  ) |>
  bind_rows() |> # mise en commun des lignes
  mutate(
    pourcentage_face = nb_faces / nb_lancers # calcul du pourcentage de réussites
  ) |> # création du graphique
  ggplot() +
  aes(x = nb_lancers, y = pourcentage_face, group = nb_lancers) +
  geom_jitter() + # ajout des points
  scale_x_log10() + # choix d'une échelle logarithmique
  theme_classic() +
  labs(
    x = "Nombre de lancers de pièce (échelle logarithmique)",
    y = "Nombre de faces (%)"
  )
)
```

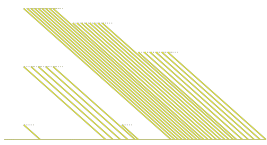


```
ggsave("img/03RL06N.png", width = 14, height = 10, units = "cm")
```

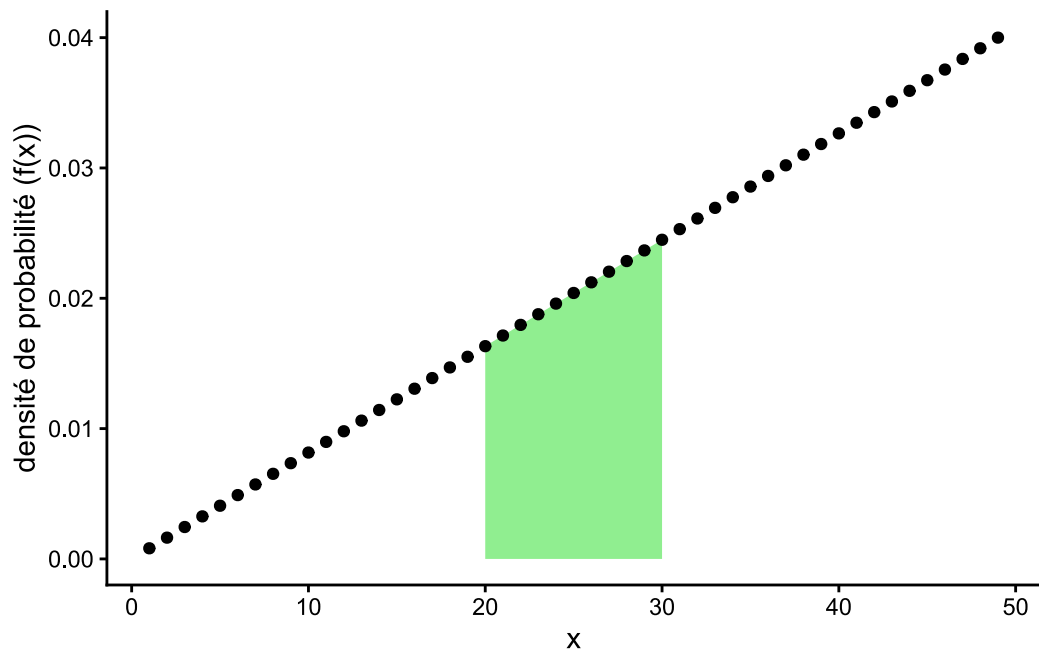
3.2 Loto biaisé

Pour illustrer, voici l'exemple d'un jeu de loto biaisé où les boules de numérotées de 1 à 49 ont un poids proportionnel à leurs chiffres donc la boule 49 aurait une masse 49 fois supérieure à la boule 1

```
# création de la table contenant les données de loto biaisé puis du graphique
tibble(
  evenement = c(1:49), # numéro de la boule 1/49
  densite = c(1:49)/sum(1:49) # probabilité de sortie
) |>
ggplot() +
  aes(
    x = evenement,
    y = densite
  ) +
  geom_polygon( # ajout du polygone vert/gris représentant la probabilité
d'obtenir une boule ayant un numéro supérieur ou égale à 20 mais inférieure ou
égale à 30
    data = tibble(
      x = c(20, 20, 30, 30),
      y = c(0, 20/sum(1:49), 30/sum(1:49), 0)
    )
  )
```



```
),  
  aes(x = x, y = y),  
  fill = "light green"  
) +  
  geom_point() + # ajout de la probabilité de chaque boule de sortir  
  theme_classic() +  
  xlab("x") +  
  ylab("densité de probabilité (f(x))")
```

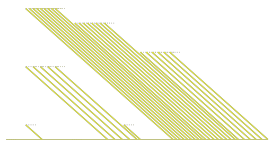


```
ggsave("img/03Rl07N.png", width = 14, height = 10, units = "cm")
```

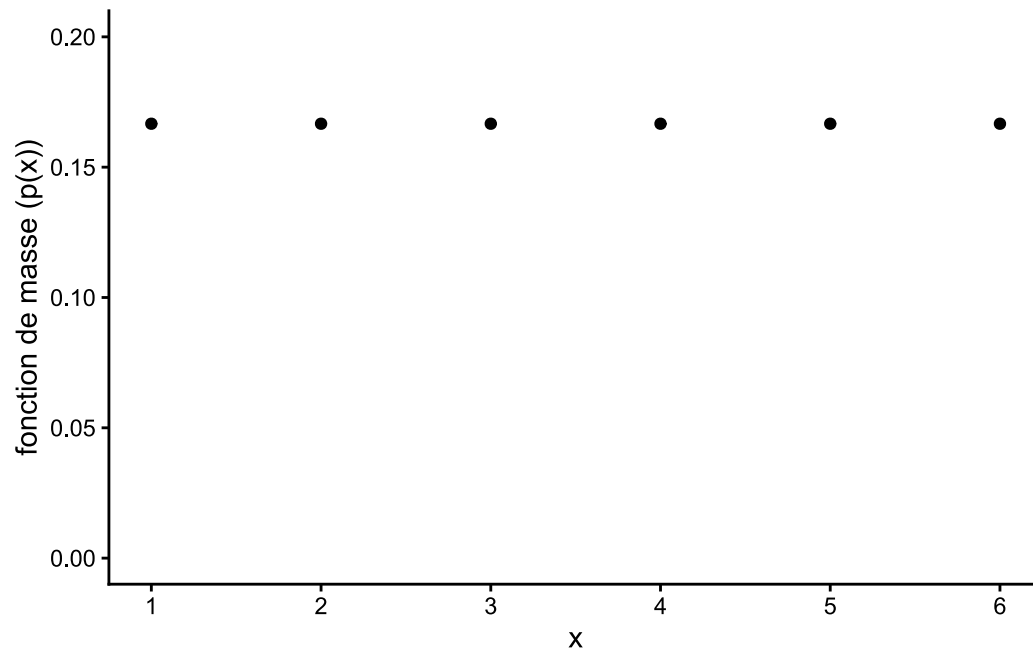
3.3 Loi uniforme

3.3.a pour un dé équilibré

```
tibble(  
  evenement = c(1:6),  
  probabilite = 1/6  
) |>  
  ggplot() +  
  aes(  
    x = evenement,  
    y = probabilite  
  ) +
```



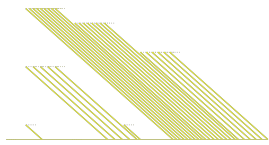
```
geom_point() +  
theme_classic() +  
scale_x_continuous(breaks = c(1:6)) +  
coord_cartesian(ylim = c(0, 0.2)) +  
xlab("x") +  
ylab("fonction de masse (p(x))")
```



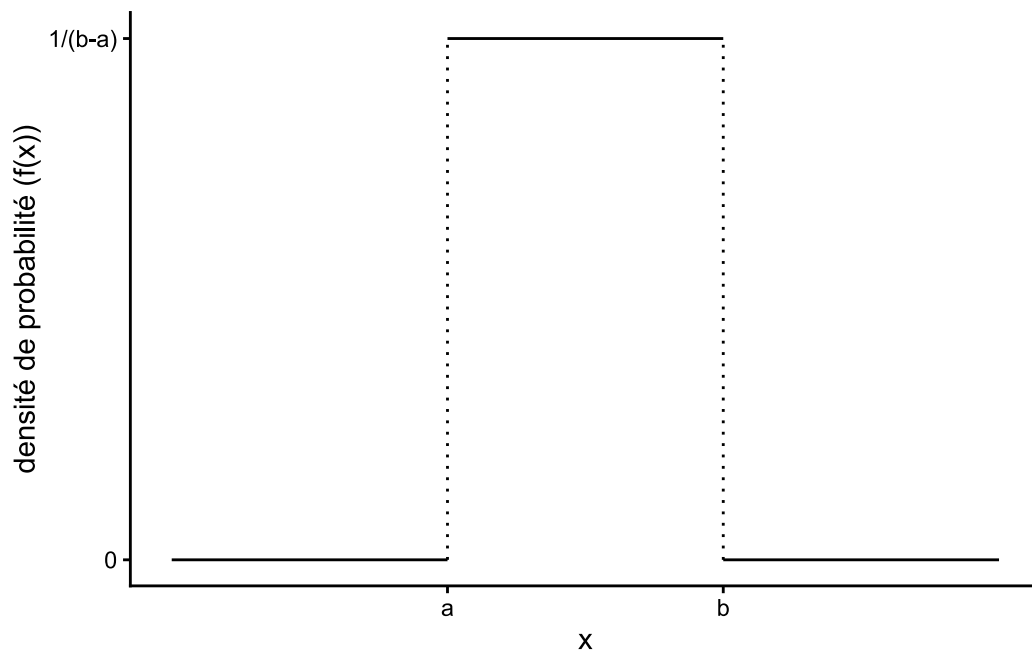
```
ggsave("img/03Rl08N.png", width = 14, height = 10, units = "cm")
```

3.3.b loi de densité uniforme continue

```
tibble(  
  evenement = c(0:1),  
  densite = 1  
) |>  
ggplot() +  
aes(  
  x = evenement,  
  y = densite  
) +  
annotate("segment", x = -1, y = 0, xend = 0, yend = 0) +  
annotate("segment", x = 0, y = 1, xend = 1, yend = 1) +  
annotate("segment", x = 1, y = 0, xend = 2, yend = 0) +
```



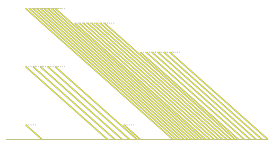
```
annotate("segment", x = 0, y = 0, xend = 0, yend = 1, linetype = "dotted") +  
annotate("segment", x = 1, y = 0, xend = 1, yend = 1, linetype = "dotted") +  
theme_classic() +  
scale_x_continuous(breaks = c(0,1), labels = c("a", "b")) +  
scale_y_continuous(breaks = c(0, 1), labels = c("0", "1/(b-a)")) +  
labs(  
  x = "x",  
  y = "densité de probabilité (f(x))"  
)
```



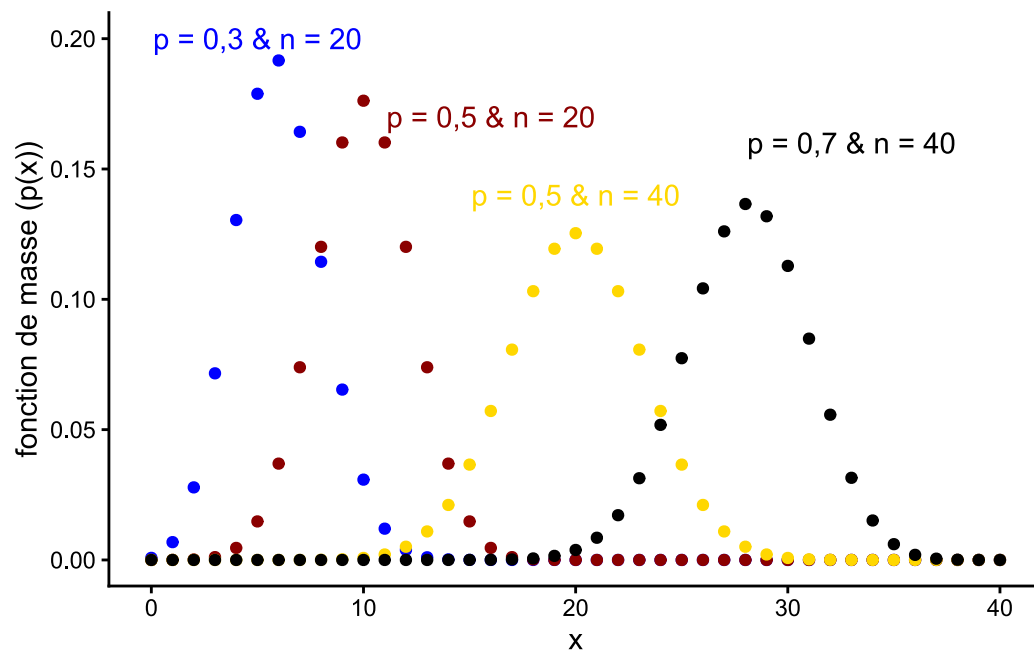
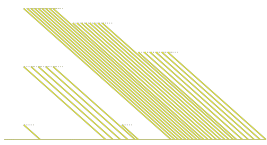
```
ggsave("img/03Rl09N.png", width = 14, height = 8, units = "cm")
```

3.4 Loi Binomiale

```
# choix de représenter entre 0 et 40  
intervalle <- (0:40)  
  
# loi binomiale avec p = 0.3, 0.5 ou 0.7 et pour 20 ou 40 épreuves de Bernoulli  
tibble(  
  loi_binomiale =  
    rep(  
      c(  
        "p = 0,3 & n = 20",
```



```
      "p = 0,5 & n = 20",
      "p = 0,5 & n = 40",
      "p = 0,7 & n = 40"
    ),
    each = length(intervalle)
  ),
  x = rep(intervalle, 4),
  densite =
    c(
      dbinom(intervalle, prob = 0.3, size = 20),
      dbinom(intervalle, prob = 0.5, size = 20),
      dbinom(intervalle, prob = 0.5, size = 40),
      dbinom(intervalle, prob = 0.7, size = 40)
    )
) |>
ggplot() +
  aes(x = x, y = densite, color = loi_binomiale) +
  geom_point() +
  theme_classic() +
  scale_color_manual(values = c("blue", "darkred", "gold", "black")) +
  annotate("text", x = 5, y = 0.2, label = "p = 0,3 & n = 20", color = "blue") +
  annotate("text", x = 16, y = 0.17, label = "p = 0,5 & n = 20", color =
"darkred") +
  annotate("text", x = 20, y = 0.14, label = "p = 0,5 & n = 40", color = "gold") +
  annotate("text", x = 33, y = 0.16, label = "p = 0,7 & n = 40") +
  theme(legend.position = "none") +
  labs(
    y = "fonction de masse (p(x))"
  )
```

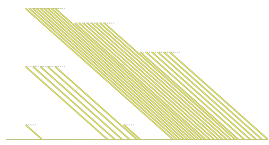


```
ggsave("img/03Rl10N.png", width = 14, height = 8, units = "cm")
```

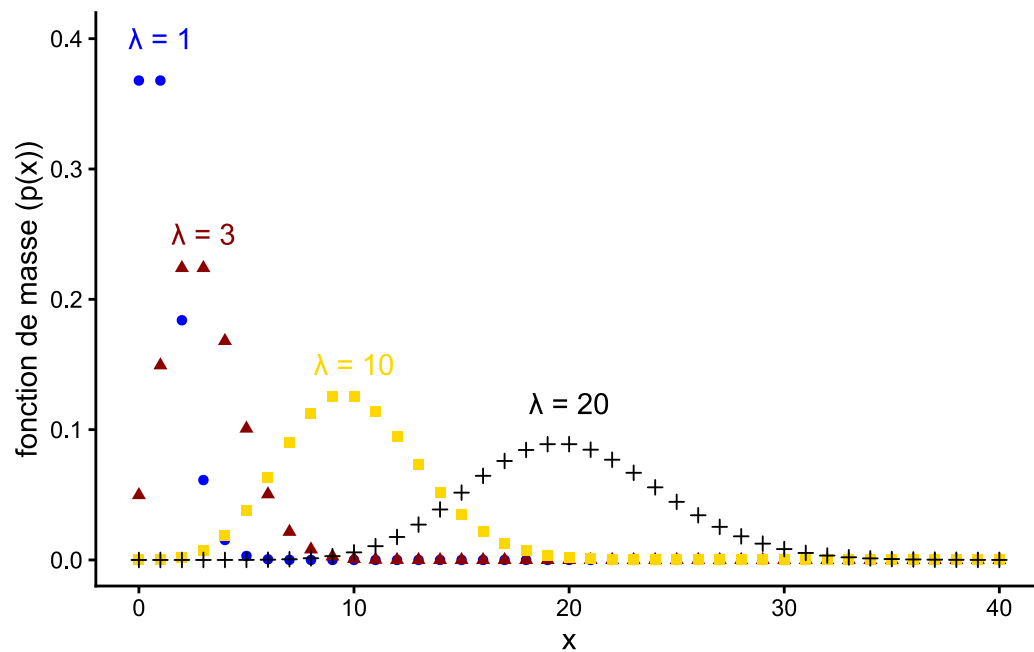
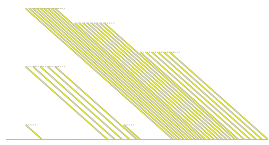
3.5 Loi de Poisson

```
# choix de représenter entre 0 et 40
intervalle <- (0:40)

# loi de poisson de paramètre λ égale à 1, 3, 10 et 20
tibble(
  loi_de_poisson = factor(
    rep(
      c(
        "λ = 1",
        "λ = 3",
        "λ = 10",
        "λ = 20"
      ),
      each = length(intervalle)
    ),
    levels = c(
      "λ = 1",
      "λ = 3",
      "λ = 10",
      "λ = 20"
    )
  )
)
```

```
)  
)  
,  
x = rep(intervalle, 4),  
densite =  
  c(  
    dpois(intervalle, lambda = 1),  
    dpois(intervalle, lambda = 3),  
    dpois(intervalle, lambda = 10),  
    dpois(intervalle, lambda = 20))  
) |>  
ggplot() +  
aes(  
  x = x,  
  y = densite,  
  color = loi_de_poisson,  
  shape = loi_de_poisson  
) +  
geom_point() +  
theme_classic() +  
scale_color_manual(values = c("blue", "darkred", "gold", "black")) +  
annotate("text", x = 1, y = 0.4, label = " $\lambda = 1$ ", color = "blue") +  
annotate("text", x = 3, y = 0.25, label = " $\lambda = 3$ ", color = "darkred") +  
annotate("text", x = 10, y = 0.15, label = " $\lambda = 10$ ", color = "gold") +  
annotate("text", x = 20, y = 0.12, label = " $\lambda = 20$ ") +  
ylab("fonction de masse (p(x))") +  
theme(legend.position = "none")
```



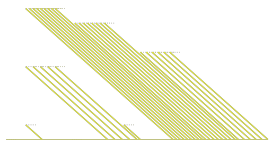
```
ggsave("img/03Rl11N.png", width = 14, height = 8, units = "cm")
```

3.6 Loi normale

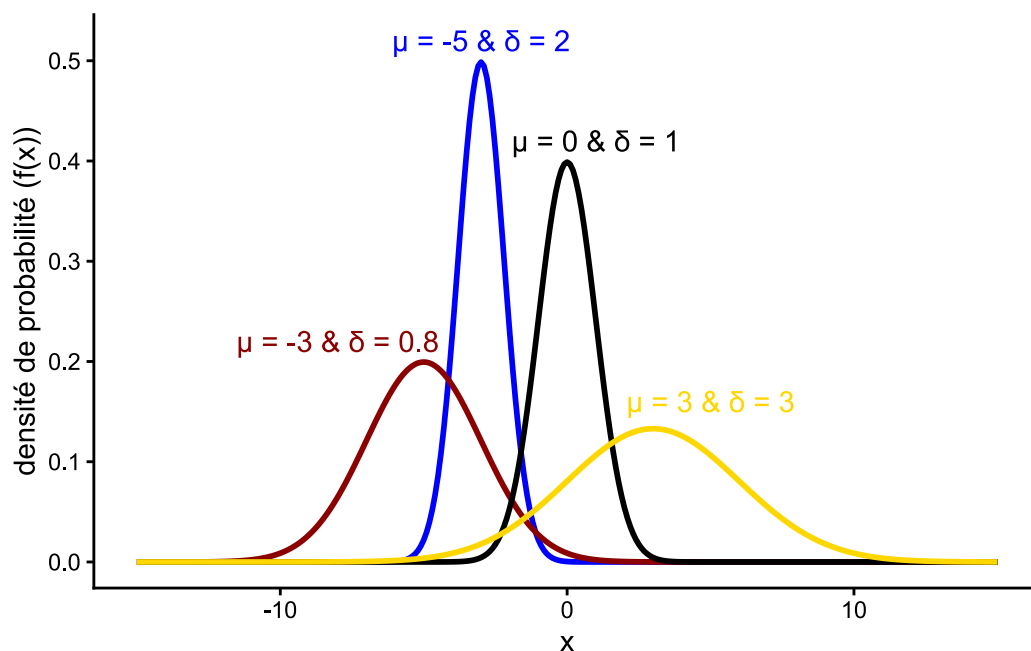
```
# choix de l'intervalle de représentation
intervalle <- seq(-15, 15, 0.1)

# loi normale de paramètres  $\mu$  égale à -5, -3, 0 ou 3 et  $\sigma$  égale à 2, 0.8, 1 ou 3
tibble(
  loi_normale =
    rep(
      c(
        " $\mu = 0$  &  $\sigma = 1$ ",
        " $\mu = 3$  &  $\sigma = 3$ ",
        " $\mu = -3$  &  $\sigma = 0.8$ ",
        " $\mu = -5$  &  $\sigma = 2$ "
      ),
      each = length(intervalle)
    ),
  x = rep(intervalle, 4),
  densite =
    c(
      dnorm(intervalle, mean = 0, sd = 1),
      dnorm(intervalle, mean = 3, sd = 3),

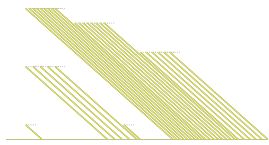
```



```
dnorm(intervalle, mean = -3, sd = 0.8),
dnorm(intervalle, mean = -5, sd = 2))
) |>
ggplot() +
aes(x = x, y = densite, color = loi_normale) +
geom_line(linewidth = 1) +
theme_classic() +
scale_color_manual(values = c("blue", "darkred", "black", "gold")) +
annotate("text", x = -8, y = 0.22, label = " $\mu = -3$  &  $\sigma = 0.8$ ", color =
"darkred") +
  annotate("text", x = -3, y = 0.52, label = " $\mu = -5$  &  $\sigma = 2$ ", color =
"blue") +
  annotate("text", x = 1, y = 0.42, label = " $\mu = 0$  &  $\sigma = 1$ ") +
  annotate("text", x = 5, y = 0.16, label = " $\mu = 3$  &  $\sigma = 3$ ", color =
"gold") +
ylab("densité de probabilité (f(x))") +
theme(legend.position = "none")
```



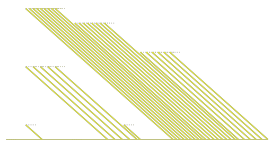
```
ggsave("img/03Rl12N.png", width = 14, height = 8, units = "cm")
```



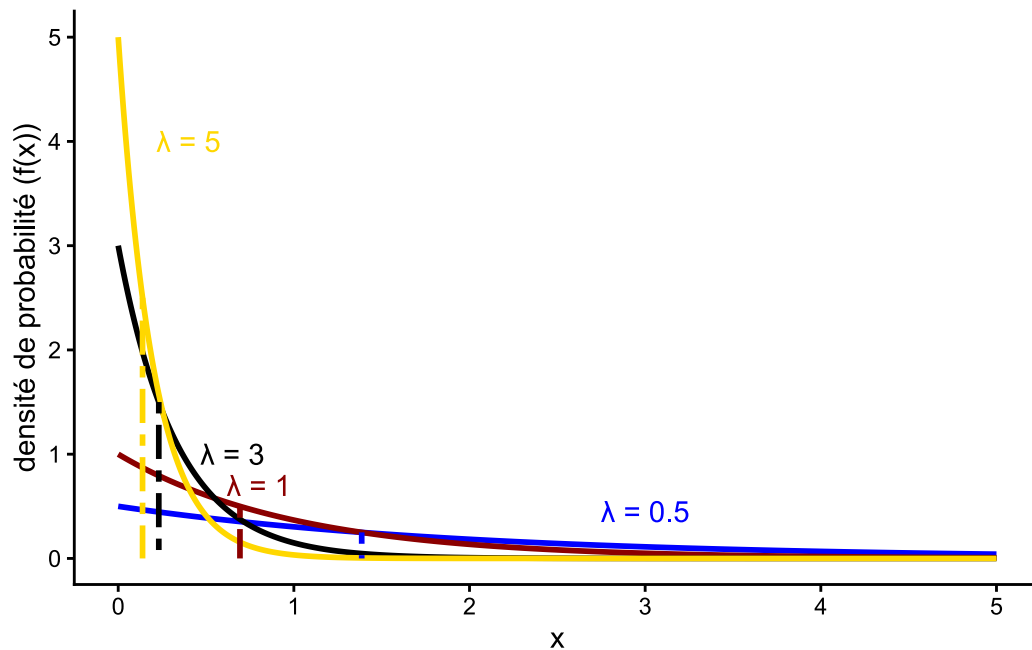
3.7 Loi exponentielle

```
# choix de l'intervalle de représentation
intervalle <- seq(0, 5, 0.01)

# loi exponentielle de paramètre  $\lambda$  égal à 0.5, 1, 3 et 5
tibble(
  loi_expo =
    rep(
      c(
        " $\lambda = 0.5$ ",
        " $\lambda = 1$ ",
        " $\lambda = 3$ ",
        " $\lambda = 5$ "
      ),
      each = length(intervalle)
    ),
  x = rep(intervalle, 4),
  densite =
    c(
      dexp(intervalle, rate = 0.5),
      dexp(intervalle, rate = 1),
      dexp(intervalle, rate = 3),
      dexp(intervalle, rate = 5)
    )
) |>
ggplot() +
  aes(x = x, y = densite, color = loi_expo) +
  geom_line(linewidth = 1) +
  annotate("text", x = 0.4, y = 4, label = " $\lambda = 5$ ", color = "gold") +
  annotate("text", x = 0.65, y = 1, label = " $\lambda = 3$ ") +
  annotate("text", x = 0.8, y = 0.7, label = " $\lambda = 1$ ", color = "darkred") +
  annotate("text", x = 3, y = 0.45, label = " $\lambda = 0.5$ ", color = "blue") +
  annotate("segment", x = log(2) * 1/5, y = dexp(log(2) * 1/5, rate = 5), xend =
log(2) * 1/5, yend = 0, linetype = "twodash", color = "gold", linewidth = 1) +
  annotate("segment", x = log(2) * 1/3, y = dexp(log(2) * 1/3, rate = 3), xend =
log(2) * 1/3, yend = 0, linetype = "twodash", color = "black", linewidth = 1) +
  theme_classic() +
  annotate("segment", x = log(2), y = dexp(log(2), rate = 1), xend = log(2),
yend = 0, linetype = "twodash", color = "dark red", linewidth = 1) +
  theme_classic() +
  annotate("segment", x = log(2) * 2, y = dexp(log(2) * 2, rate = 0.5), xend =
log(2) * 2, yend = 0, linetype = "twodash", color = "blue", linewidth = 1) +
  theme_classic()
```



```
scale_color_manual(values = c("blue", "dark red", "black", "gold")) +
ylab("densité de probabilité (f(x))") +
theme(legend.position = "none")
```



```
ggsave("img/03Rl13N.png", width = 14, height = 8, units = "cm")
```

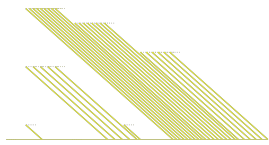
4 Statistiques descriptives univariées

4.1 Type de données

```
classe_fictive <-
  tibble(
    groupe_eleve = rep(1:3, 10),
    nom_eleve = glue::glue("Nom_{1:30}"),
    age_eleve = rnorm(n = 30, mean = 9, sd = 0.5) |> round()
  )

glimpse(classe_fictive)
```

```
Rows: 30
Columns: 3
$ groupe_eleve <int> 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2...
```



```
$ nom_eleve <glue> "Nom_1", "Nom_2", "Nom_3", "Nom_4", "Nom_5", "Nom_6", "No...
$ age_eleve <dbl> 9, 9, 10, 9, 10, 9, 9, 9, 9, 9, 9, 8, 9, 9, 9, 9, 8, 9, 9,...
```

```
summary(classe_fictive)
```

groupe_eleve	nom_eleve	age_eleve
Min. :1	Length :30	Min. : 8.0
1st Qu.:1	N.unique :30	1st Qu.: 9.0
Median :2	N.blank : 0	Median : 9.0
Mean :2	Min.nchar: 5	Mean : 9.1
3rd Qu.:3	Max.nchar: 6	3rd Qu.: 9.0
Max. :3		Max. :10.0

modification de la nature des données.

```
classe_fictive <-
  mutate(
    .data = classe_fictive,
    groupe_eleve = groupe_eleve |> as_factor(),
    nom_eleve = nom_eleve |> as.character()
  )

glimpse(classe_fictive)
```

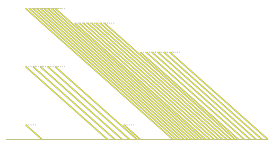
```
Rows: 30
Columns: 3
$ groupe_eleve <fct> 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2...
$ nom_eleve <chr> "Nom_1", "Nom_2", "Nom_3", "Nom_4", "Nom_5", "Nom_6", "Nom...
$ age_eleve <dbl> 9, 9, 10, 9, 10, 9, 9, 9, 9, 9, 9, 8, 9, 9, 9, 9, 8, 9, 9,...
```

4.2 variable qualitative

Il est conseillé d'aller voir la page d'aide sur le jeu de données avec ?mpg.

```
# type de transmission (f = traction, r = propulsion, 4 = 4 roues motrices)
mpg |> count(drv)
```

```
# A tibble: 3 × 2
  drv      n
<chr> <int>
1 4      103
```

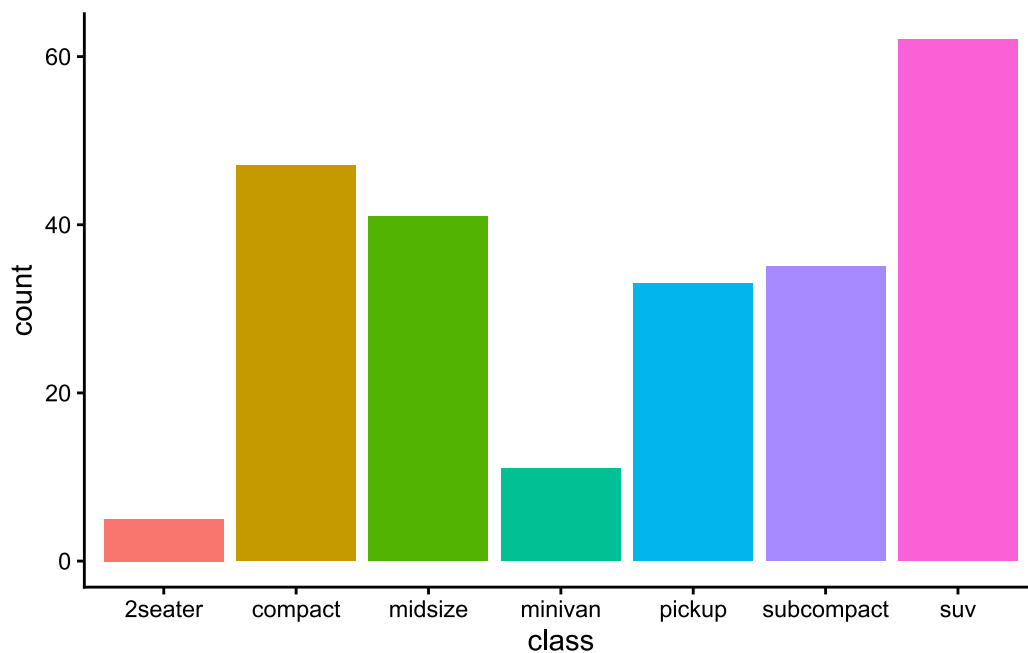


```
2 f      106
3 r       25
```

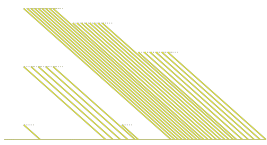
```
# ajout d'une colonne pour calculer les proportions de chaque catégories
mpg |>
  count(manufacturer) |>
  mutate(prop = round(n*100/nrow(mpg))) |>
  arrange(desc(n)) |>
  write_csv(
    file = "data/table_resume_constructeur.csv"
  )
```

4.2.a création d'un graphique en barres

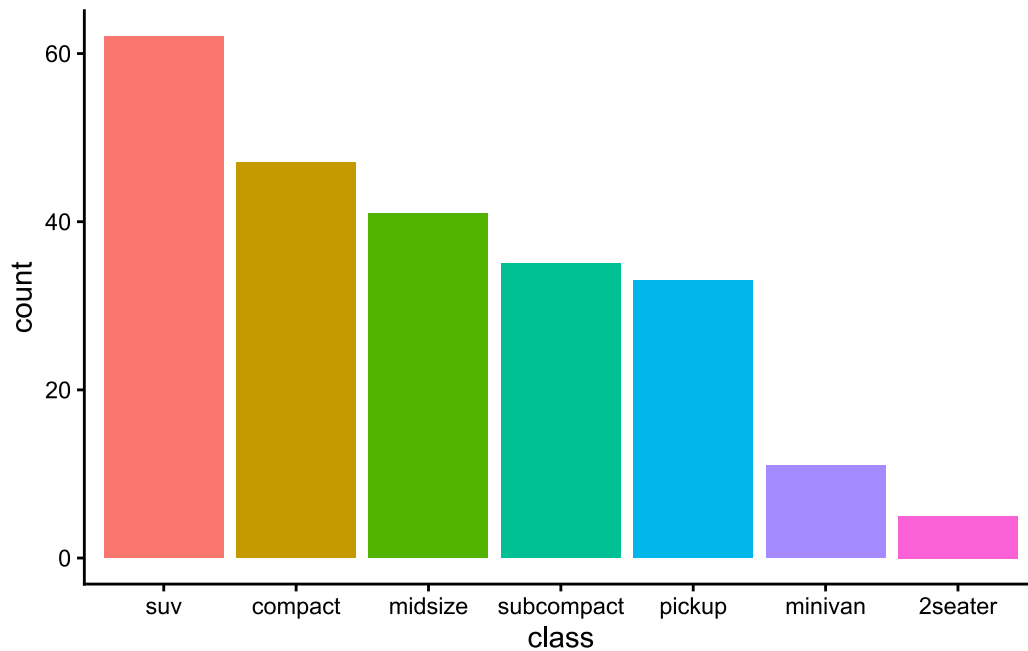
```
mpg |>
  ggplot() +
  aes(x = class, fill = class) +
  geom_bar() +
  theme_classic() +
  theme(legend.position = "none")
```



```
ggsave("img/03Rl18N.png", width = 14, height = 10, units = "cm")
```

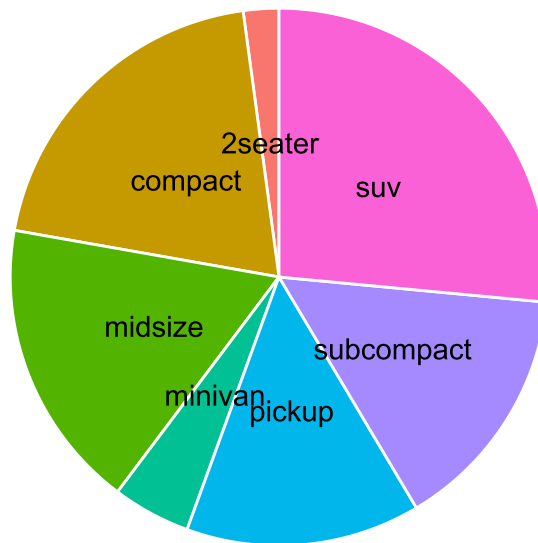
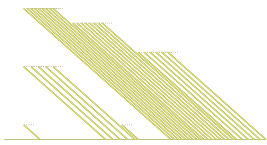


```
mpg |>
  ggplot() +
  aes(x = class |> fct_infreq(), fill = class |> fct_infreq()) +
  labs(x = "class") +
  geom_bar() +
  theme_classic() +
  theme(legend.position = "none")
```



4.2.b création d'un graphique circulaire

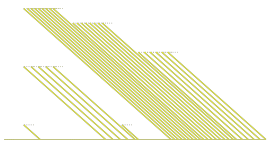
```
mpg |>
  count(class) |>
  ggplot() +
  aes(x = "", y = n, fill = class) +
  geom_bar(stat = "identity", color = "white") +
  geom_text(aes(label = class), position = position_stack(vjust = 0.5)) +
  coord_polar("y") +
  theme_void() +
  theme(legend.position = "none")
```

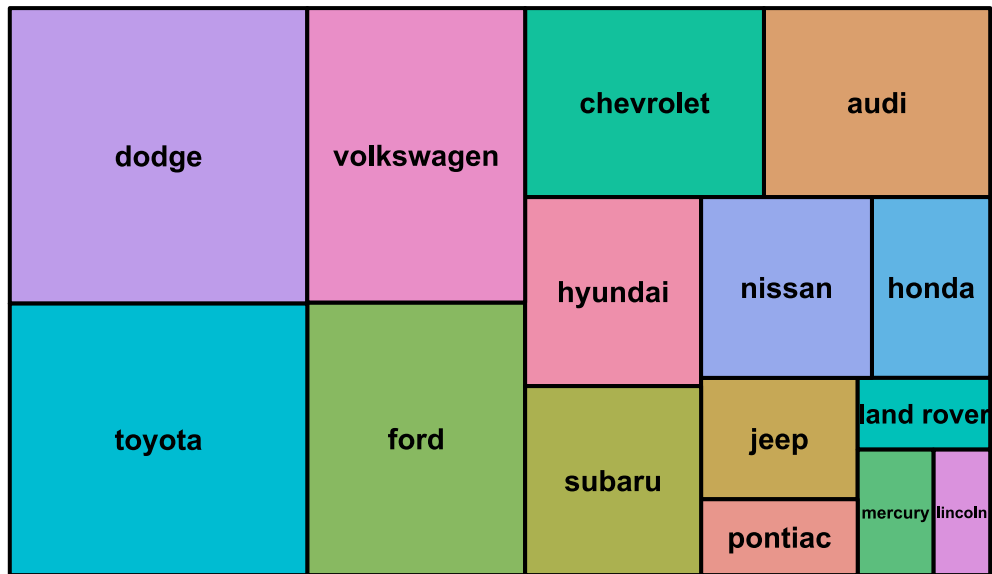
```
ggsave("img/03Rl19N.png", width = 10, height = 10, units = "cm")
```

4.2.c treemap

```
library(treemap)
treemap(
  mpg |>
    count(manufacturer),
  index = "manufacturer",
  vSize = "n",
  title = "Les constructeurs"
)
```



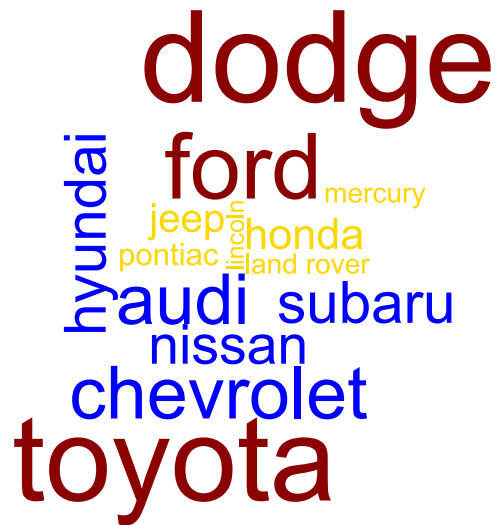
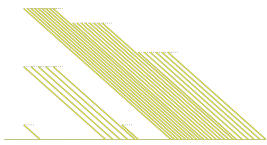
Les constructeurs



4.2.d nuage de mots

```
# création du tableau
tableau_constructeur <- mpg |>
  count(manufacturer)

# création du nuage de mots
wordcloud::wordcloud(
  words = tableau_constructeur$manufacturer,
  freq = tableau_constructeur$n,
  colors = c("gold", "blue", "dark red")
)
```



```
Warning in wordcloud::wordcloud(words = tableau_construteur$manufacturer, :  
volkswagen could not be fit on page. It will not be plotted.
```

4.3 variable quantitative

4.3.a valeurs de dispersion

```
summary(mpg$cty)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
9.00	14.00	17.00	16.86	19.00	35.00

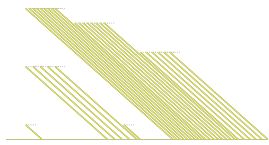
```
min(mpg$cty)
```

```
[1] 9
```

```
median(mpg$cty)
```

```
[1] 17
```

```
mean(mpg$cty)
```



```
[1] 16.85897
```

```
max(mpg$cty)
```

```
[1] 35
```

```
sd(mpg$cty)
```

```
[1] 4.255946
```

```
var(mpg$cty)
```

```
[1] 18.11307
```

```
#calcul de l'étendue  
range(mpg$cty) |> diff()
```

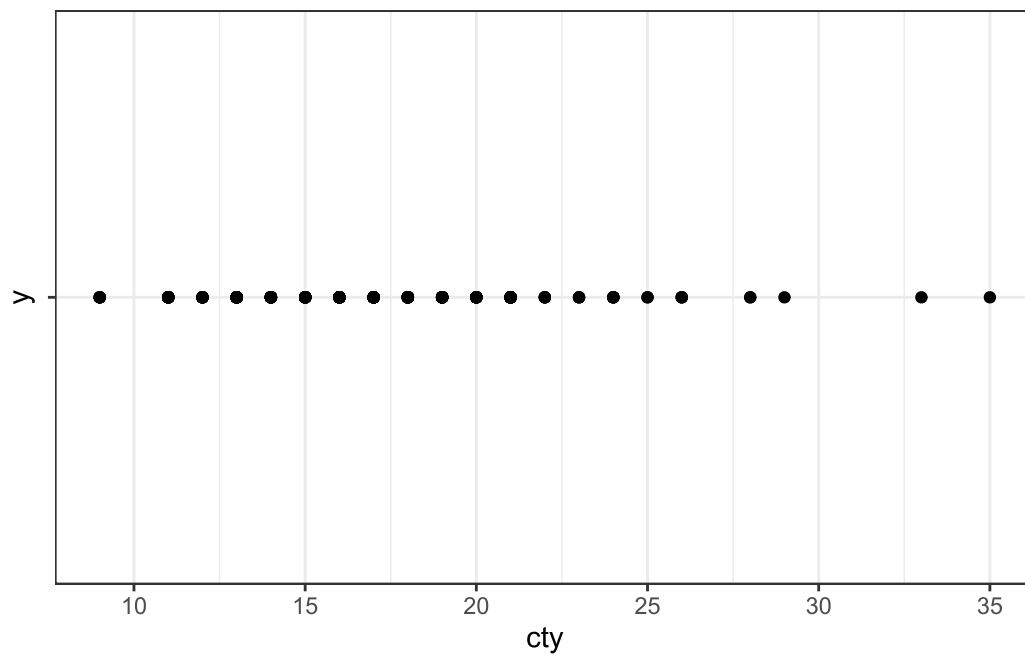
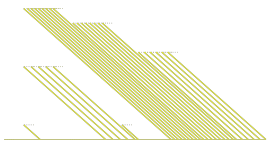
```
[1] 26
```

```
max(mpg$cty) - min(mpg$cty)
```

```
[1] 26
```

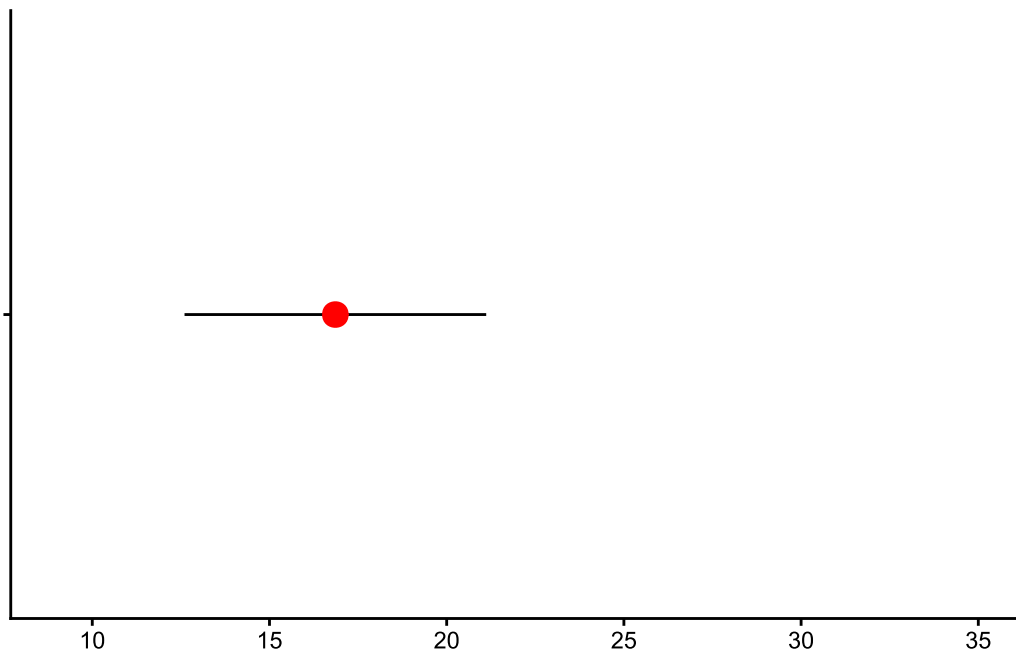
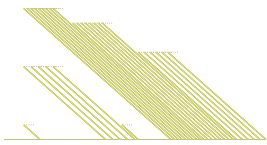
4.3.b nuage de points

```
ggplot(  
  data = mpg  
) +  
  aes(x = cty, y = "") +  
  geom_point() +  
  theme_bw()
```



4.3.c données de positionnement

```
mpg |>
  ggplot() +
  aes(x = cty, y = "") +
  annotate("segment", x = mean(mpg$cty) + sd(mpg$cty), xend = mean(mpg$cty) -
sd(mpg$cty), y = "", yend = "") +
  annotate("point", x = mean(mpg$cty), y = "", color = "red", size = 4) +
  xlim(9,35) +
  theme_classic()
```

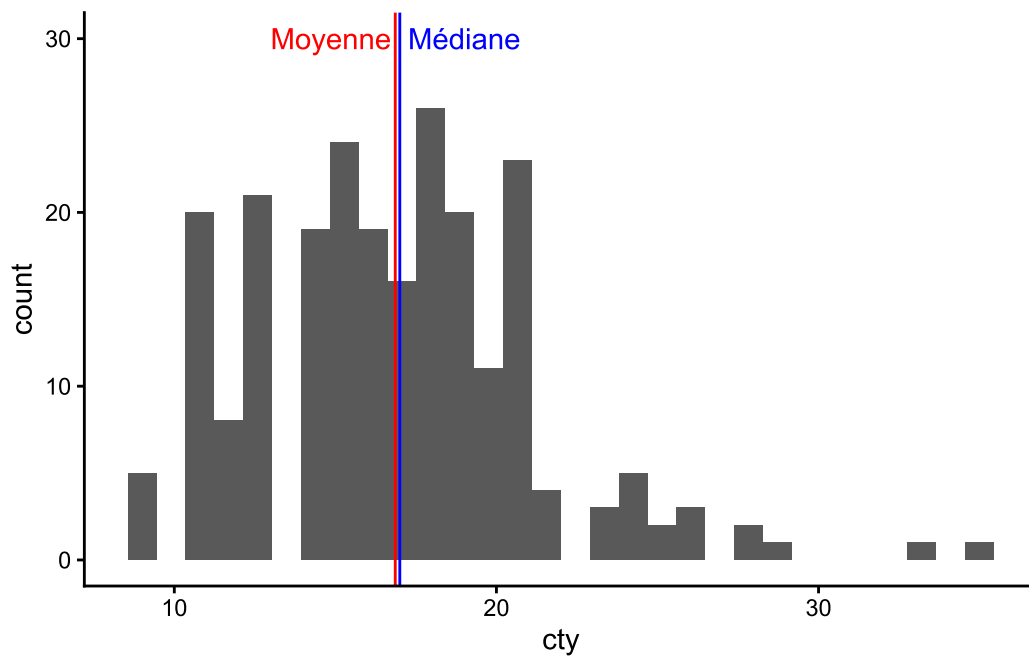
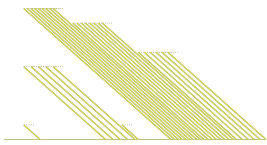


```
ggsave("img/03RL24N.png", width = 14, height = 8, units = "cm")
```

4.3.d histogramme avec la moyenne et la médiane

```
ggplot(  
  data = mpg  
) +  
  aes(x = cty) +  
  geom_histogram() +  
  geom_vline(aes(xintercept = mean(cty)), color = "red") +  
  geom_vline(aes(xintercept = median(cty)), color = "blue") +  
  annotate("text", x = mean(mpg$cty) - 2, y = 30, label = "Moyenne", color =  
"red") +  
  annotate("text", x = median(mpg$cty) + 2, y = 30, label = "Médiane", color =  
"blue") +  
  theme_classic()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

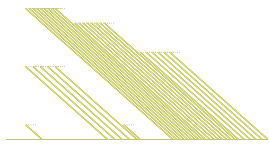


```
ggsave("img/03RL25N.png", width = 14, height = 8, units = "cm")
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

4.3.e Nuage de points avec la moyenne et la médiane

```
ggplot(iris) +  
  aes(x = Petal.Length, y = "") +  
  geom_point() +  
  geom_point(aes(mean(Petal.Length)), color = "red", size = 4) +  
  geom_point(aes(median(Petal.Length)), color = "blue", size = 4) +  
  geom_text(  
    aes(  
      mean(Petal.Length),  
      label = "Moyenne"  
    ),  
    nudge_y = 0.5,  
    color = "red"  
  ) +  
  geom_text(  
    aes(  
      median(Petal.Length),  
      label = "Médiane"  
    )
```



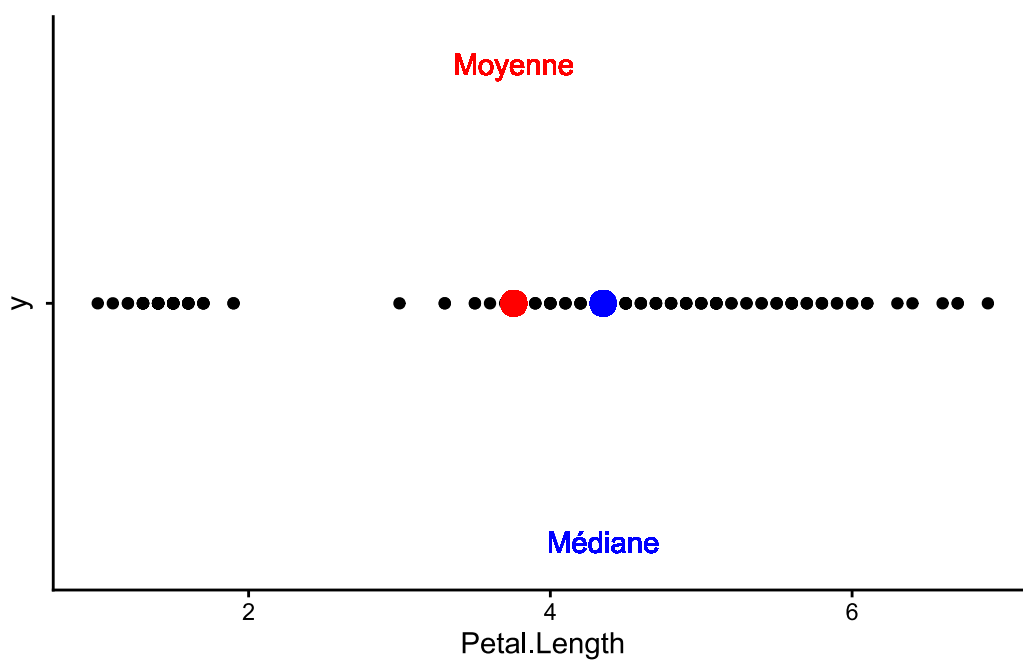
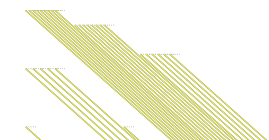
```
    ),  
    nudge_y = - 0.5,  
    color = "blue"  
  ) +  
  theme_classic()
```

Warning in `geom_point(aes(mean(Petal.Length)), color = "red", size = 4)`: All aesthetics have length 1, but the data has 150 rows.
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

Warning in `geom_point(aes(median(Petal.Length)), color = "blue", size = 4)`: All aesthetics have length 1, but the data has 150 rows.
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

Warning in `geom_text(aes(mean(Petal.Length), label = "Moyenne"), nudge_y = 0.5, :`
All aesthetics have length 1, but the data has 150 rows.
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

Warning in `geom_text(aes(median(Petal.Length), label = "Médiane"), nudge_y = -0.5, :`
All aesthetics have length 1, but the data has 150 rows.
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

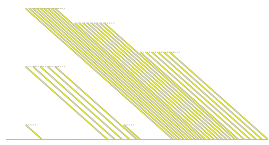


4.3.f nuage de point dispersé

```
ggplot(  
  data = iris  
) +  
  aes(x = Petal.Length, y = "") +  
  geom_jitter() +  
  geom_point(aes(mean(Petal.Length)), color = "red", size = 4) +  
  geom_point(aes(median(Petal.Length)), color = "blue", size = 4) +  
  geom_text(aes(mean(Petal.Length), label = "Moyenne"), nudge_y = 0.5, color =  
"red") +  
  geom_text(aes(median(Petal.Length), label = "Médiane"), nudge_y = - 0.5, color  
= "blue") +  
  theme_classic()
```

Warning in `geom_point(aes(mean(Petal.Length)), color = "red", size = 4)`: All aesthetics have length 1, but the data has 150 rows.
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

Warning in `geom_point(aes(median(Petal.Length)), color = "blue", size = 4)`: All aesthetics have length 1, but the data has 150 rows.



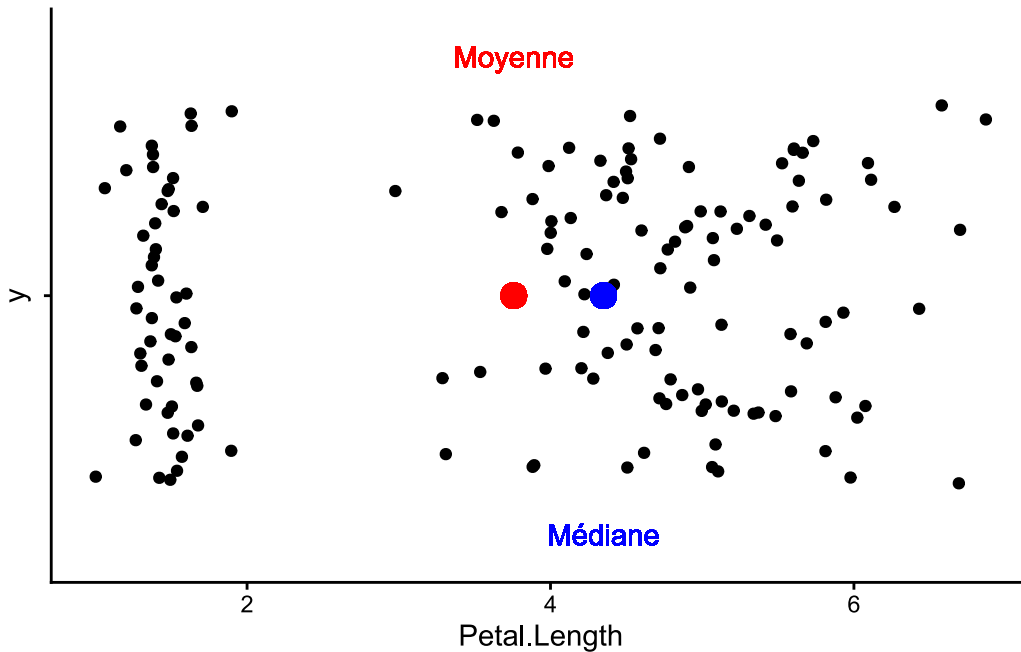
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

Warning in `geom_text(aes(mean(Petal.Length), label = "Moyenne"), nudge_y = 0.5, :`
All aesthetics have length 1, but the data has 150 rows.

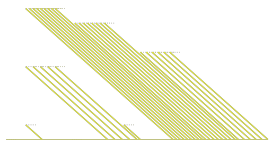
i Please consider using ``annotate()`` or provide this layer with data containing a single row.

Warning in `geom_text(aes(median(Petal.Length), label = "Médiane"), nudge_y =`
-0.5, : All aesthetics have length 1, but the data has 150 rows.

i Please consider using ``annotate()`` or provide this layer with data containing a single row.



```
iris |>
  count(Petal.Length) |>
  ggplot() +
  aes(x = Petal.Length, y = "", size = n) +
  geom_point() +
  geom_point(aes(weighted.mean(Petal.Length, n), y = ""), color = "red",
inherit.aes = FALSE) +
  geom_point(aes(matrixStats::weightedMedian(Petal.Length, n), y = ""), color =
"blue", inherit.aes = FALSE) +
```



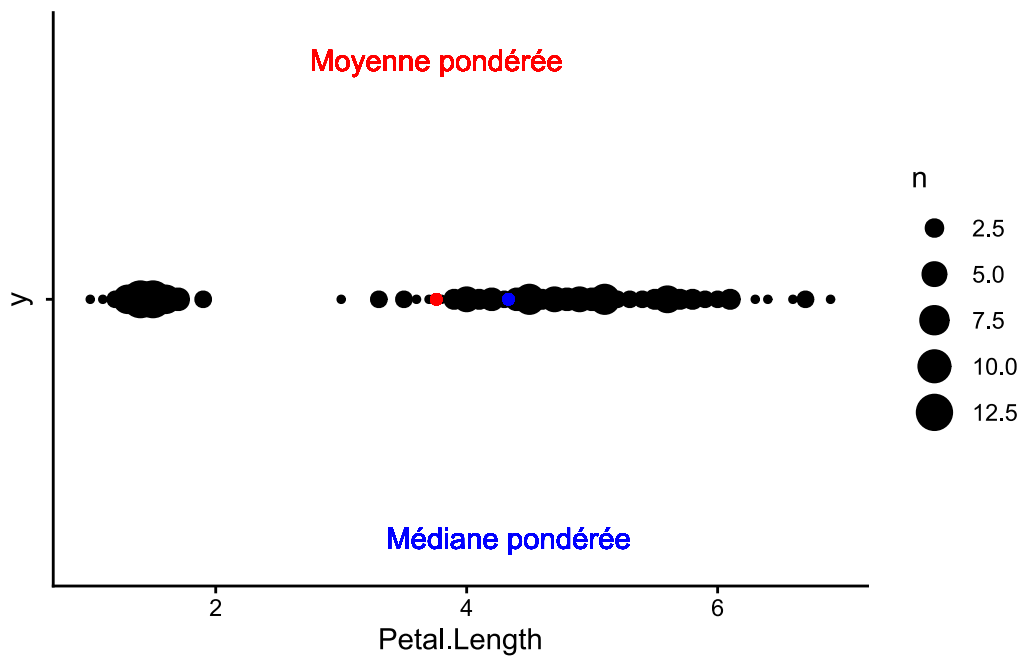
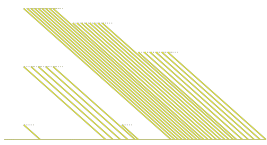
```
geom_text(aes(weighted.mean(Petal.Length, n), y = "", label = "Moyenne
pondérée"), nudge_y = 0.5, color = "red", inherit.aes = FALSE) +
geom_text(aes(matrixStats::weightedMedian(Petal.Length, n), y = "", label =
"Médiane pondérée"), nudge_y = - 0.5, color = "blue", inherit.aes = FALSE) +
theme_classic()
```

Warning in geom_point(aes(weighted.mean(Petal.Length, n), y = ""), color = "red", : All aesthetics have length 1, but the data has 43 rows.
i Please consider using `annotate()` or provide this layer with data containing a single row.

Warning in geom_point(aes(matrixStats::weightedMedian(Petal.Length, n), : All aesthetics have length 1, but the data has 43 rows.
i Please consider using `annotate()` or provide this layer with data containing a single row.

Warning in geom_text(aes(weighted.mean(Petal.Length, n), y = "", label = "Moyenne pondérée"), : All aesthetics have length 1, but the data has 43 rows.
i Please consider using `annotate()` or provide this layer with data containing a single row.

Warning in geom_text(aes(matrixStats::weightedMedian(Petal.Length, n), y = "", : All aesthetics have length 1, but the data has 43 rows.
i Please consider using `annotate()` or provide this layer with data containing a single row.



4.3.g quantiles

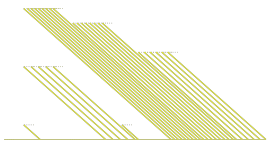
```
quantile(x = mpg$cty)
```

0%	25%	50%	75%	100%
9	14	17	19	35

```
# déciles
quantile(
  x = mpg$cty,
  probs = seq(0, 1, 0.1)
)
```

0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
9	11	13	14	15	17	18	19	20	21	35

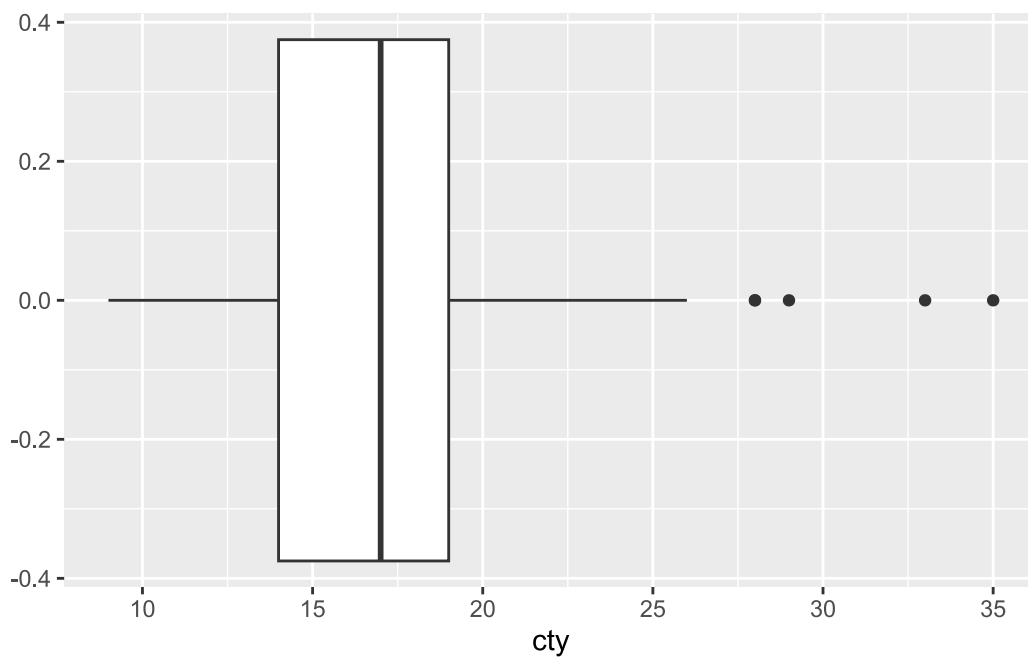
```
# centiles
centile <- quantile(
  x = mpg$cty,
  probs = seq(0, 1, 0.01)
)
centile
```

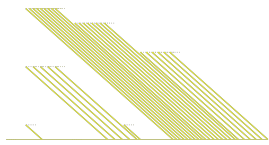


0%	1%	2%	3%	4%	5%	6%	7%	8%	9%	10%	11%	12%
9.00	9.00	10.32	11.00	11.00	11.00	11.00	11.00	11.00	11.00	11.00	12.00	12.00
13%	14%	15%	16%	17%	18%	19%	20%	21%	22%	23%	24%	25%
12.00	12.62	13.00	13.00	13.00	13.00	13.00	13.00	13.00	13.00	13.59	14.00	14.00
26%	27%	28%	29%	30%	31%	32%	33%	34%	35%	36%	37%	38%
14.00	14.00	14.00	14.00	14.00	14.23	15.00	15.00	15.00	15.00	15.00	15.00	15.00
39%	40%	41%	42%	43%	44%	45%	46%	47%	48%	49%	50%	51%
15.00	15.00	15.00	16.00	16.00	16.00	16.00	16.00	16.00	16.00	16.00	17.00	17.00
52%	53%	54%	55%	56%	57%	58%	59%	60%	61%	62%	63%	64%
17.00	17.00	17.00	17.00	17.00	18.00	18.00	18.00	18.00	18.00	18.00	18.00	18.00
65%	66%	67%	68%	69%	70%	71%	72%	73%	74%	75%	76%	77%
18.00	18.00	18.00	19.00	19.00	19.00	19.00	19.00	19.00	19.00	19.00	19.08	20.00
78%	79%	80%	81%	82%	83%	84%	85%	86%	87%	88%	89%	90%
20.00	20.00	20.00	20.73	21.00	21.00	21.00	21.00	21.00	21.00	21.00	21.00	21.00
91%	92%	93%	94%	95%	96%	97%	98%	99%	100%			
22.00	22.00	23.00	24.00	24.00	24.68	26.00	26.68	28.67	35.00			

4.3.h boîte à moustaches

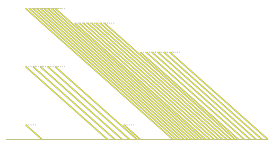
```
# simple
ggplot(data = mpg) +
  geom_boxplot(aes(cty))
```



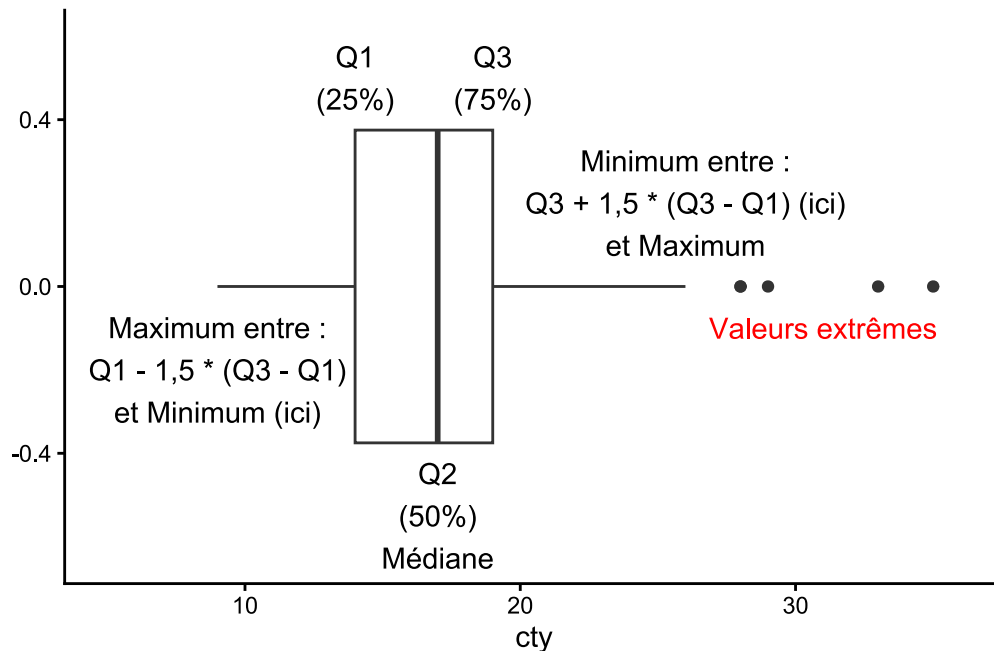


```
# avec commentaires
quartile <- quantile(mpg$cty)

ggplot(data = mpg) +
  geom_boxplot(aes(cty)) +
  annotate(
    "text",
    x = quartile[1],
    y = - 0.2 ,
    label = "Maximum entre :\nQ1 - 1,5 * (Q3 - Q1)\net Minimum (ici)"
  ) +
  annotate(
    "text",
    x = quartile[2],
    y = 0.5,
    label = "Q1\n(25%)"
  ) +
  annotate(
    "text",
    x = quartile[3],
    y = - 0.55,
    label = "Q2\n(50%)\nMédiane"
  ) +
  annotate(
    "text",
    x = quartile[4],
    y = 0.5,
    label = "Q3\n(75%)"
  ) +
  annotate(
    "text",
    x = 26,
    y = 0.2,
    label = "Minimum entre :\nQ3 + 1,5 * (Q3 - Q1) (ici)\net Maximum"
  ) +
  annotate(
    "text",
    x = 31,
    y = -0.1,
    label = "Valeurs extrêmes",
    color = "red"
  ) +
  xlim(5, 36) +
  ylim(-0.65, 0.6) +
```

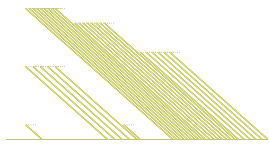


```
ylab("") +  
theme_classic()
```

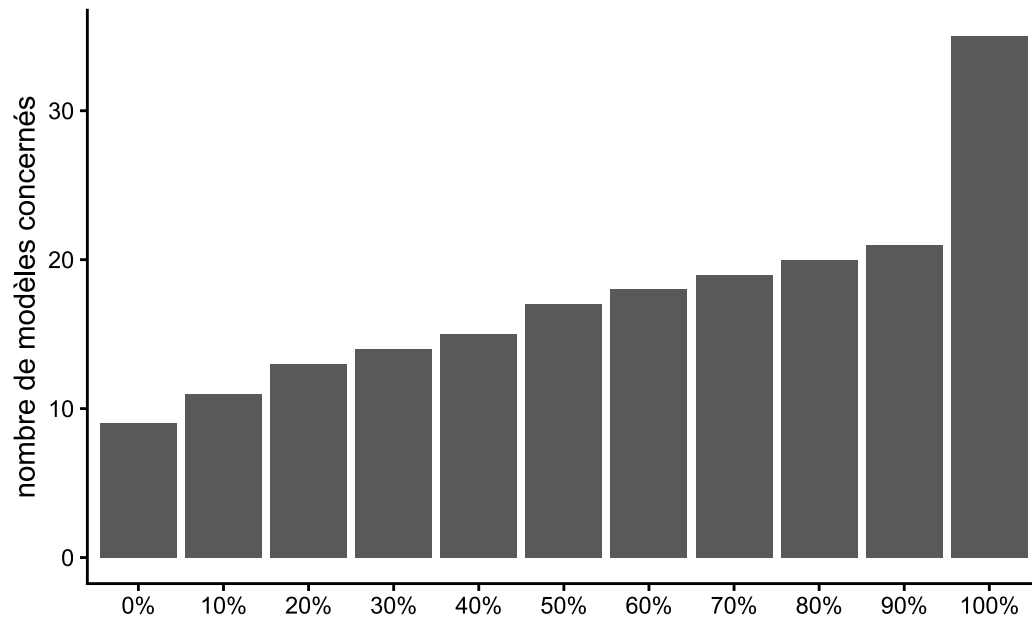


4.3.i visualisation des deciles

```
cty_decile <- quantile(  
  x = mpg$cty,  
  probs = seq(0, 1, 0.1)  
)  
  
tibble(  
  pourcentage = names(cty_decile),  
  cty_deciles = cty_decile  
) |>  
ggplot() +  
aes(  
  x = pourcentage |> fct_inorder(),  
  y = cty_decile  
) +  
geom_bar(stat = "identity") +  
labs(  
  x = "",  
  y = "nombre de modèles concernés"
```



```
) +  
theme_classic()
```



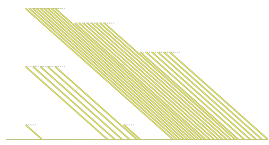
```
ggsave("img/03Rl31N.png", width = 14, height = 8, units = "cm")
```

4.3.j filtrer sur les valeurs extremes

```
mpg |>  
  filter(  
    cty > 26  
  )
```

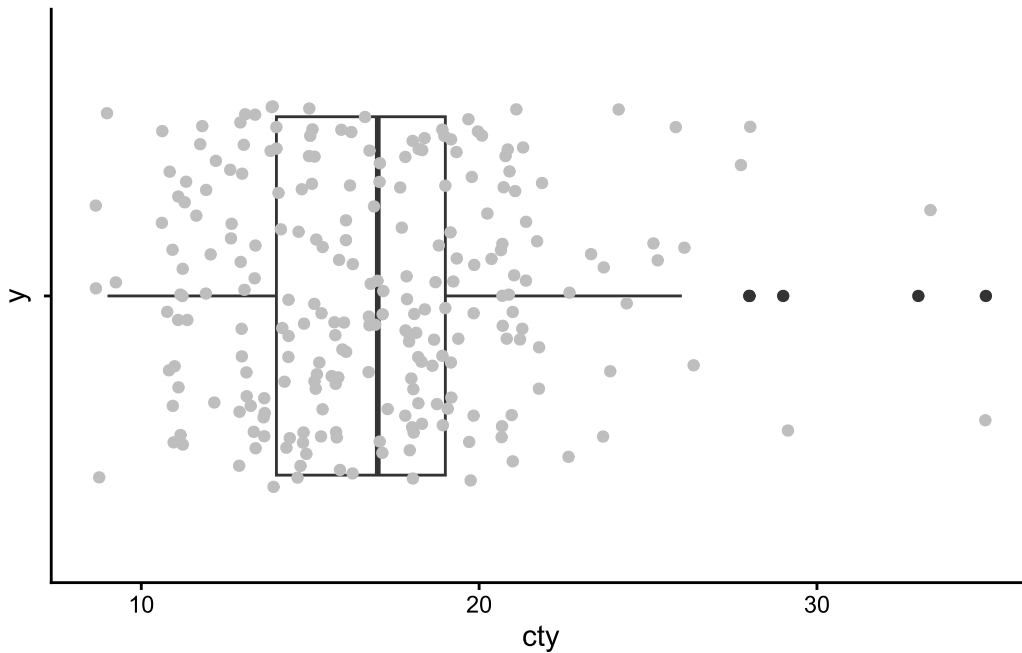
```
# A tibble: 5 × 11
```

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
	<chr>	<chr>	<dbl>	<int>	<int>	<chr>	<chr>	<int>	<int>	<chr>	<chr>
1	honda	civic	1.6	1999	4	manua...	f	28	33	r	subc...
2	toyota	corolla	1.8	2008	4	manua...	f	28	37	r	comp...
3	volkswagen	jetta	1.9	1999	4	manua...	f	33	44	d	comp...
4	volkswagen	new beetle	1.9	1999	4	manua...	f	35	44	d	subc...
5	volkswagen	new beetle	1.9	1999	4	auto(...	f	29	41	d	subc...



4.3.k boîte à moustaches avec points dispersés

```
ggplot(data = mpg) +  
  aes(x= cty, y = "") +  
  geom_boxplot() +  
  geom_jitter(color = "grey") +  
  theme_classic()
```



5 Statistiques descriptives bivariées

5.1 2 variables quantitatives

5.1.a test de Pearson

```
cor.test(mpg$cty, mpg$hwy)
```

Pearson's product-moment correlation

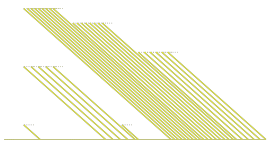
data: mpg\$cty and mpg\$hwy

t = 49.585, df = 232, p-value < 2.2e-16

alternative hypothesis: true correlation is not equal to 0

95 percent confidence interval:

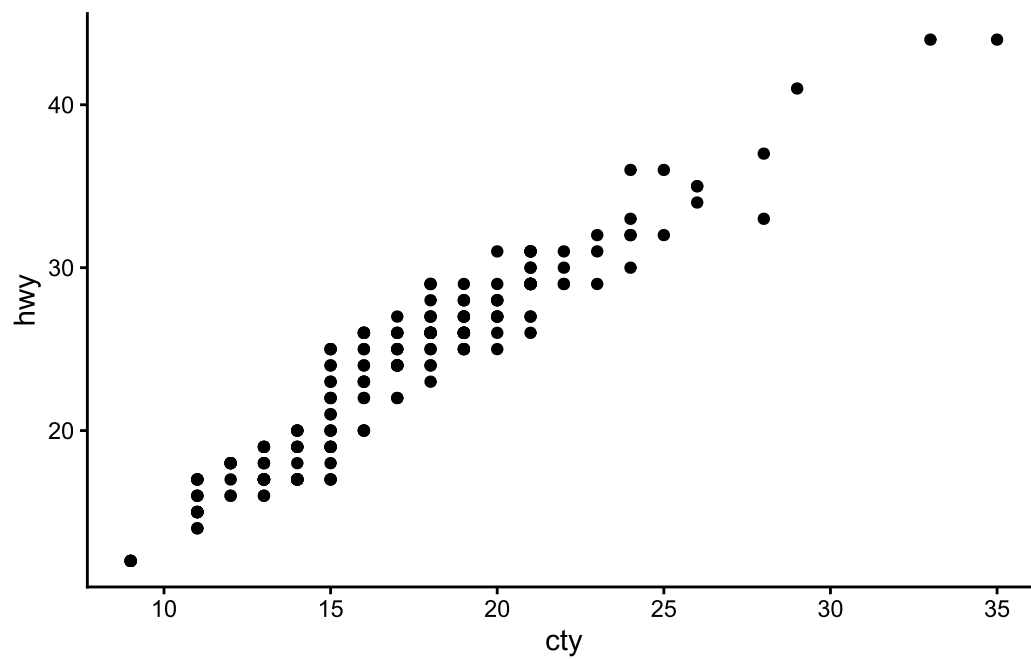
0.9433129 0.9657663



```
sample estimates:
      cor 
0.9559159
```

5.1.b graphique

```
ggplot(mpg) +
  aes(
    x = cty,
    y = hwy
  ) +
  geom_point() +
  theme_classic()
```

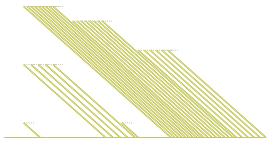


5.2 Une quanti et une quali

5.2.a test du Khi-2

```
chisq.test(mpg$cty, mpg$class)
```

```
Warning in chisq.test(mpg$cty, mpg$class): L'approximation du Chi-2 est
peut-être incorrecte
```

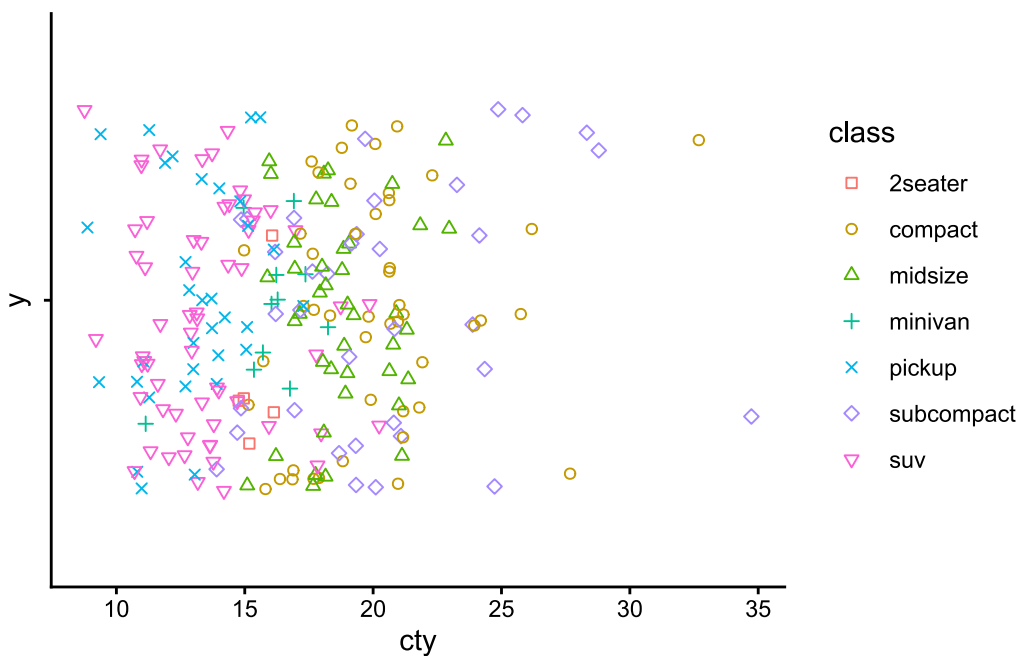


Pearson's Chi-squared test

```
data: mpg$cty and mpg$class  
X-squared = 293.17, df = 120, p-value < 2.2e-16
```

5.2.b représentation graphique avec forme et couleur

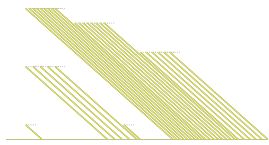
```
ggplot(mpg) +  
  aes(x = cty, y = "", color = class, shape = class) +  
  geom_jitter() +  
  scale_shape_manual(values = c(0:6)) +  
  theme_classic()
```



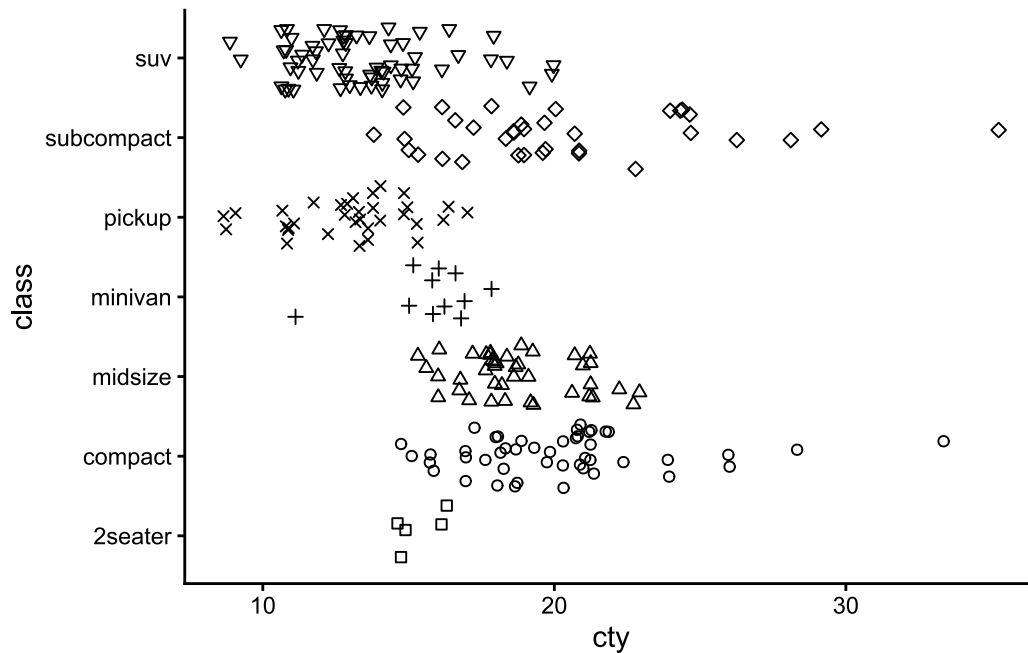
```
ggsave("img/03Rl35N.png", width = 14, height = 8, units = "cm")
```

5.2.c une ligne par type de véhicule

```
ggplot(mpg) +  
  aes(x = cty, y = class, shape = class) +  
  geom_jitter() +  
  scale_shape_manual(values = c(0:6)) +
```



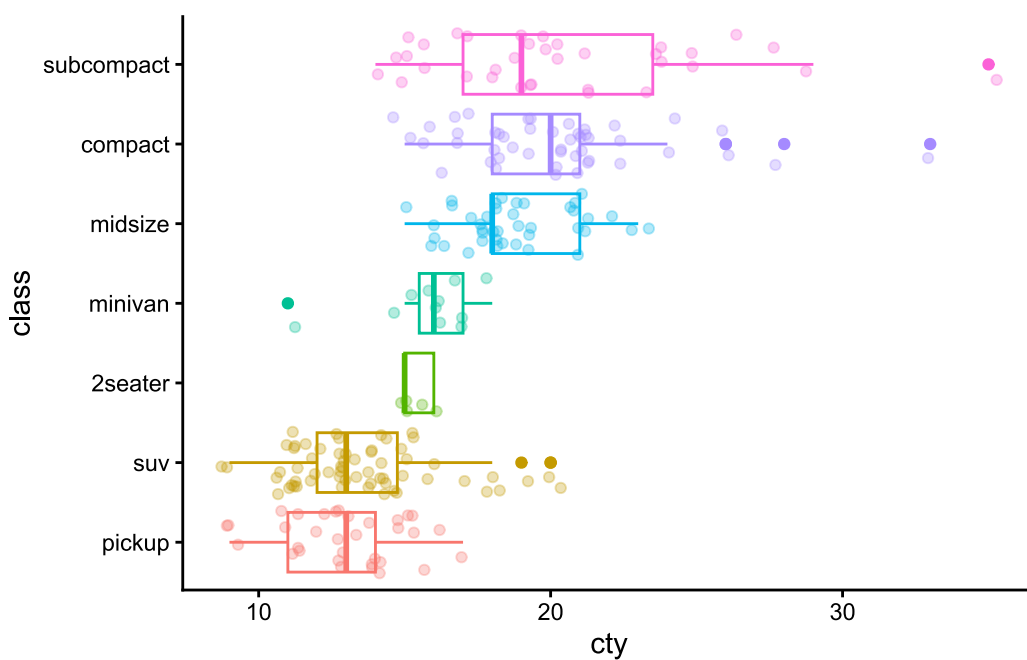
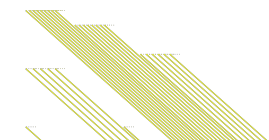
```
theme_classic() +  
theme(legend.position = "none")
```



```
ggsave("img/03Rl36N.png", width = 14, height = 8, units = "cm")
```

5.2.d avec des boîtes à moustaches

```
ggplot(mpg) +  
  aes(x = cty, y = reorder(class, cty), color = reorder(class, cty)) +  
  geom_boxplot() +  
  geom_jitter(alpha = 0.3) +  
  labs(  
    y = "class",  
    color = "class"  
  ) +  
  theme_classic() +  
  theme(legend.position = "none")
```



```
ggsave("img/03RL37N.png", width = 14, height = 8, units = "cm")
```

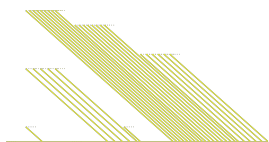
5.3 2 variables qualitatives

5.3.a tableau de contingence

```
mpg |>
  count(manufacturer, class) |>
  pivot_wider(names_from = class, values_from = n)
```

A tibble: 15 × 8

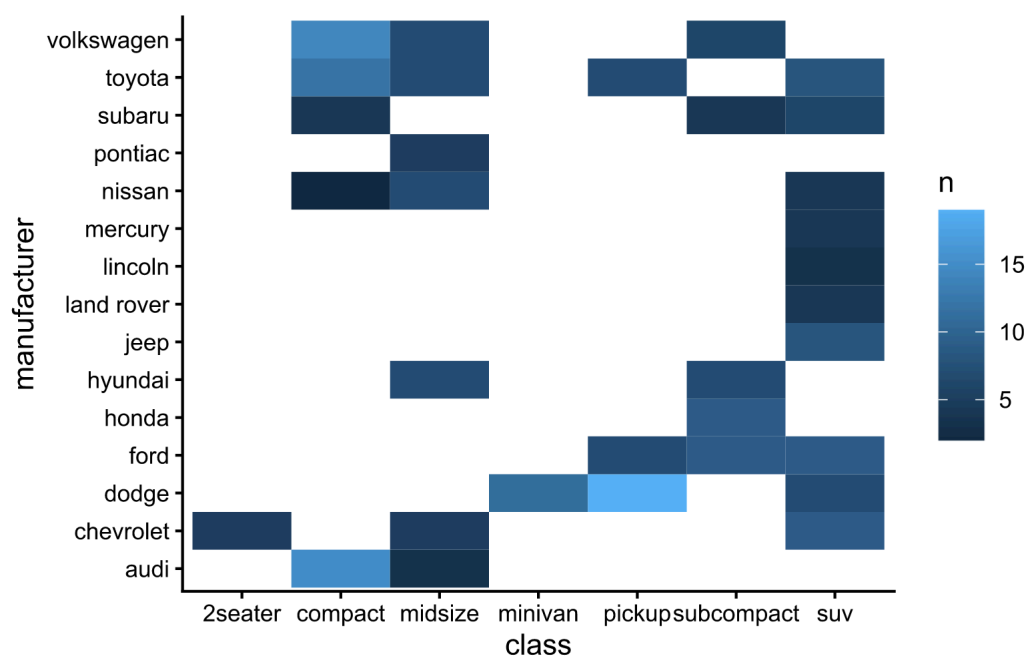
	manufacturer	compact	midsize	`2seater`	suv	minivan	pickup	subcompact
	<chr>	<int>	<int>	<int>	<int>	<int>	<int>	<int>
1	audi	15	3	NA	NA	NA	NA	NA
2	chevrolet	NA	5	5	9	NA	NA	NA
3	dodge	NA	NA	NA	7	11	19	NA
4	ford	NA	NA	NA	9	NA	7	9
5	honda	NA	NA	NA	NA	NA	NA	9
6	hyundai	NA	7	NA	NA	NA	NA	7
7	jeep	NA	NA	NA	8	NA	NA	NA
8	land rover	NA	NA	NA	4	NA	NA	NA
9	lincoln	NA	NA	NA	3	NA	NA	NA
10	mercury	NA	NA	NA	4	NA	NA	NA
11	nissan	2	7	NA	4	NA	NA	NA



12	pontiac	NA	5	NA	NA	NA	NA	NA
13	subaru	4	NA	NA	6	NA	NA	4
14	toyota	12	7	NA	8	NA	7	NA
15	volkswagen	14	7	NA	NA	NA	NA	6

5.3.b carte des points chauds

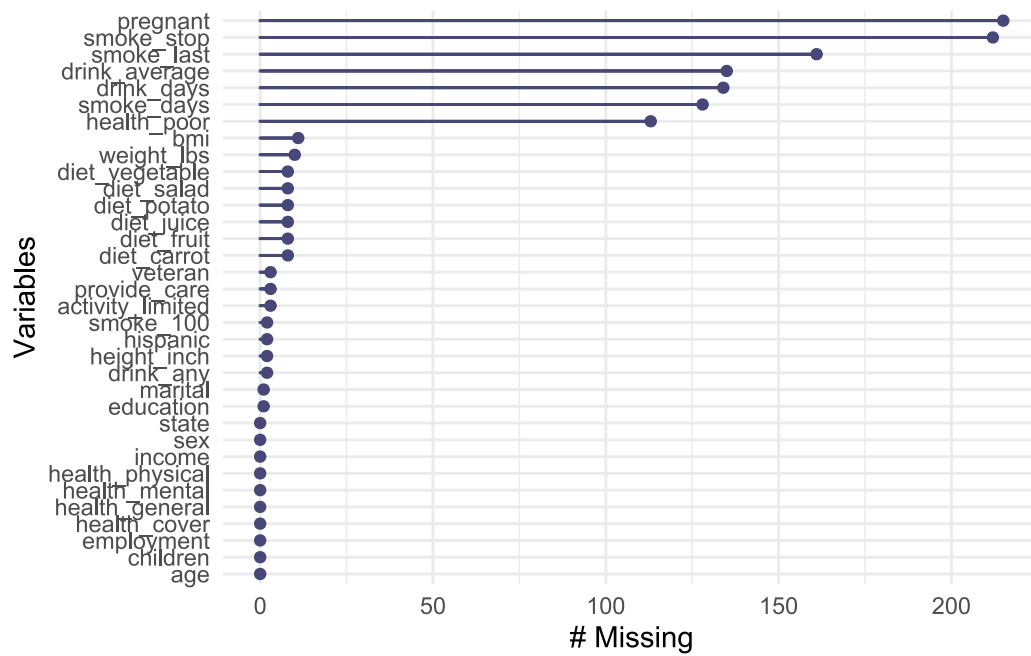
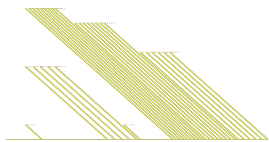
```
mpg |>
  count(manufacturer, class) |>
  ggplot() +
  aes(x = class, y = manufacturer, fill = n) +
  geom_tile() +
  theme_classic()
```



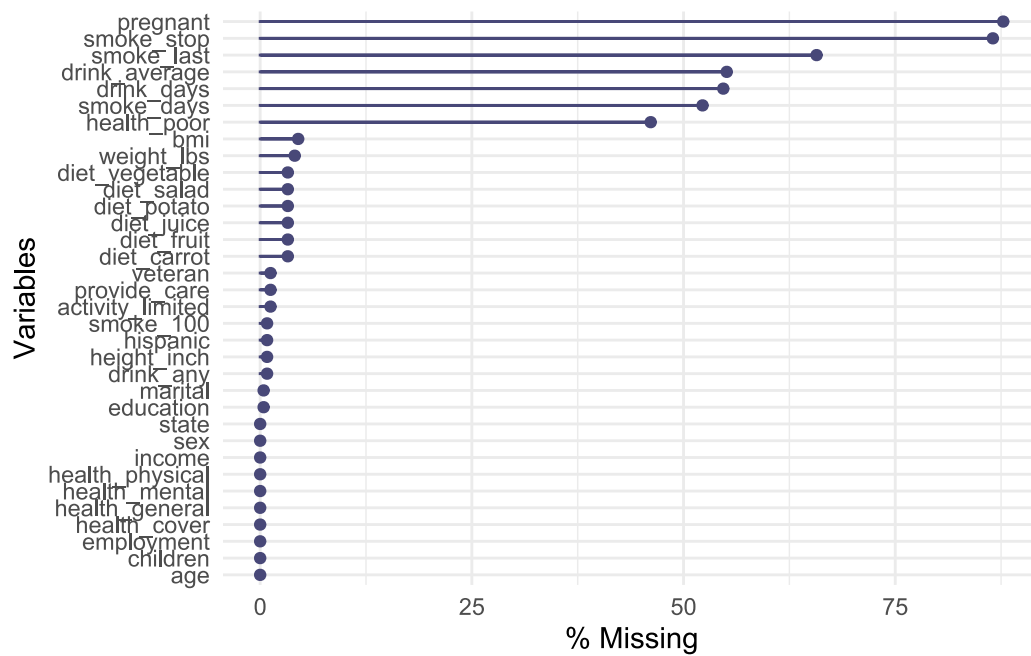
6 Valeurs manquantes

```
library(naniar)

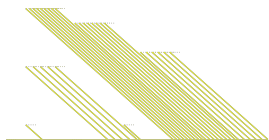
gg_miss_var(riskfactors)
```



```
gg_miss_var(riskfactors, show_pct = TRUE)
```



```
gg_miss_upset(riskfactors)
```



Warning: ``aes_string()`` was deprecated in ggplot2 3.0.0.
 i Please use tidy evaluation idioms with ``aes()``.
 i See also ``vignette("ggplot2-in-packages")`` for more information.
 i The deprecated feature was likely used in the UpSetR package.
 Please report the issue at <https://github.com/hms-dbmi/UpSetR/issues>.

Warning: Using ``size`` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use ``linewidth`` instead.
 i The deprecated feature was likely used in the UpSetR package.
 Please report the issue at <https://github.com/hms-dbmi/UpSetR/issues>.

Warning: The ``size`` argument of ``element_line()`` is deprecated as of ggplot2 3.4.0.
 i Please use the ``linewidth`` argument instead.
 i The deprecated feature was likely used in the UpSetR package.
 Please report the issue at <https://github.com/hms-dbmi/UpSetR/issues>.

